

DJAC TOOL

Global Enterprise Compliance Intelligence Platform

Version 1.0.0 Production Release

Copyright Deposit Submission

First 25 + Last 25 Pages of Production Source Code

Submitted in accordance with U.S. Copyright Office Circular 61

In response to deposit request for computer program

Author: Ismail Ahamed Hassan Awad

Export Date: 2026-07-28

Case Number: [1-15066433901]

THREAD ID: [THREAD ID:1-6ZIZ4GW]

TECHNICAL CERTIFICATION

I hereby certify that:

- 1) This source code represents the actual production implementation of DJAC TOOL.
- 2) It was hand-coded and independently developed by Ismail Ahamed Hassan Awad.
- 3) It contains no fabricated, placeholder, or artificially generated code.
- 4) Confidential information (API keys, passwords, secrets) has been redacted.
- 5) This material is submitted for copyright registration purposes only.

Copyright (c) 2026 Ismail Ahamed Hassan Awad. All rights reserved.

```
import {
  pgEnum,
  pgTable,
  serial,
  integer,
  varchar,
  text,
  timestamp,
} from "drizzle-orm/pg-core";

//
// ENUMS
//

export const userTypeEnum = pgEnum("userType", [
  "visitor",
  "professional",
  "admin",
  "basic_user",
  "professional_user",
  "company_admin",
  "platform_admin",
  "yalla_hack_employee",
  "super_admin",
]);

export const localeEnum = pgEnum("locale", ["en", "ar", "zh"]);

export const userStatusEnum = pgEnum("userStatus", [
  "active",
  "pending",
  "suspended",
]);

export const roleEnum = pgEnum("role", [
  "user",
  "admin",
  "basic_user",
  "professional_user",
  "company_admin",
  "platform_admin",
  "yalla_hack_employee",
  "super_admin",
]);

export const userOAuthStatusEnum = pgEnum("userOAuthStatus", [
  "active",
  "invited",
  "suspended",
]);

export const accessRequestStatusEnum = pgEnum("accessRequestStatus", [
  "new",
  "reviewing",
  "approved",
  "archived",
]);

export const consultationStatusEnum = pgEnum("consultationStatus", [
  "new",
  "in_review",
  "responded",
  "closed",
]);

export const priorityEnum = pgEnum("priority", ["low", "medium", "high"]);

export const severityEnum = pgEnum("severity", [
  "low",
  "medium",
```

```

    "high",
    "critical",
  ]);

export const criticalityEnum = pgEnum("criticality", [
  "low",
  "medium",
  "high",
  "critical",
]);

export const actorTypeEnum = pgEnum("actorType", [
  "visitor",
  "client",
  "admin",
  "system",
]);

export const notificationCategoryEnum = pgEnum("notificationCategory", [
  "registration",
  "consultation",
  "assessment",
  "support",
  "system",
]);

export const relationshipTypeEnum = pgEnum("relationshipType", [
  "overlap",
  "conflict",
  "harmonization",
  "coordination",
  "gap",
  "dependency",
]);

export const mappingTypeEnum = pgEnum("mappingType", [
  "equivalent",
  "related",
  "conflicting",
  "complementary",
]);

export const assessmentStatusEnum = pgEnum("assessmentStatus", [
  "compliant",
  "partial",
  "non_compliant",
  "unknown",
]);

export const jurisdictionEnum = pgEnum("jurisdiction", [
  "China",
  "Saudi Arabia",
  "Both",
  "Other",
]);

export const planEnum = pgEnum("plan", [
  "free_trial",
  "starter",
  "professional",
  "enterprise",
]);

export const paidPlanEnum = pgEnum("paidPlan", [
  "starter",
  "professional",
  "enterprise",
]);

export const orgMemberRoleEnum = pgEnum("orgMemberRole", [
  "owner",
  "admin",
  "compliance_officer",
  "analyst",
]);

```

```

]);

export const orgMemberStatusEnum = pgEnum("orgMemberStatus", [
  "active",
  "invited",
  "suspended",
]);

export const billingIntervalEnum = pgEnum("billingInterval", [
  "monthly",
  "quarterly",
  "biannual",
  "annual",
]);

export const subscriptionStatusEnum = pgEnum("subscriptionStatus", [
  "trialing",
  "active",
  "past_due",
  "canceled",
  "incomplete",
  "paused",
]);

export const billingEventStatusEnum = pgEnum("billingEventStatus", [
  "success",
  "failed",
  "pending",
  "refunded",
]);

export const reportTypeEnum = pgEnum("reportType", [
  "full_compliance",
  "gap_analysis",
  "vendor_assessment",
  "risk_assessment",
  "executive_summary",
  "regulatory_deadline",
]);

export const reportStatusEnum = pgEnum("reportStatus", [
  "generating",
  "ready",
  "failed",
  "archived",
]);

export const auditLogCategoryEnum = pgEnum("auditLogCategory", [
  "auth",
  "data_write",
  "data_read",
  "role_change",
  "system",
  "billing",
]);

export const auditLogOutcomeEnum = pgEnum("auditLogOutcome", [
  "success",
  "failure",
  "blocked",
]);

export const taskSeverityEnum = pgEnum("taskSeverity", [
  "critical",
  "high",
  "medium",
  "low",
]);

export const taskStatusEnum = pgEnum("taskStatus", [
  "open",
  "in_progress",
  "resolved",
  "accepted_risk",

```

```

});

export const riskCategoryEnum = pgEnum("riskCategory", [
  "operational",
  "legal",
  "technical",
  "financial",
  "reputational",
]);

export const treatmentEnum = pgEnum("treatment", [
  "accept",
  "mitigate",
  "transfer",
  "avoid",
]);

export const riskStatusEnum = pgEnum("riskStatus", [
  "open",
  "in_treatment",
  "closed",
  "accepted",
]);

export const policyTypeEnum = pgEnum("policyType", [
  "policy",
  "standard",
  "procedure",
  "guideline",
]);

export const policyStatusEnum = pgEnum("policyStatus", [
  "draft",
  "under_review",
  "approved",
  "active",
  "retired",
]);

export const incidentTypeEnum = pgEnum("incidentType", [
  "data_breach",
  "unauthorized_access",
  "policy_violation",
  "system_outage",
  "third_party_breach",
  "other",
]);

export const incidentStatusEnum = pgEnum("incidentStatus", [
  "open",
  "under_investigation",
  "contained",
  "resolved",
  "closed",
]);

export const auditTypeEnum = pgEnum("auditType", [
  "internal",
  "external",
  "regulatory",
  "certification",
]);

export const auditStatusEnum = pgEnum("auditStatus", [
  "planned",
  "in_progress",
  "completed",
  "cancelled",
]);

export const recurrenceEnum = pgEnum("recurrence", [
  "none",
  "monthly",
  "quarterly",

```

```

    "biannual",
    "annual",
  ]);

export const ctemAssetTypeEnum = pgEnum("ctemAssetType", [
  "web_application",
  "api_endpoint",
  "database",
  "cloud_service",
  "network_device",
  "iot_device",
  "data_pipeline",
  "identity_provider",
  "storage_bucket",
  "other",
]);

export const regionEnum = pgEnum("region", [
  "China",
  "Saudi Arabia",
  "Cross-border",
  "Other",
]);

export const assetStatusEnum = pgEnum("assetStatus", [
  "active",
  "inactive",
  "decommissioned",
]);

export const vulnSeverityEnum = pgEnum("vulnSeverity", [
  "critical",
  "high",
  "medium",
  "low",
  "informational",
]);

export const simulationTypeEnum = pgEnum("simulationType", [
  "lateral_movement",
  "privilege_escalation",
  "data_exfiltration",
  "ransomware",
  "phishing",
  "api_abuse",
  "supply_chain",
  "ddos",
  "insider_threat",
  "other",
]);

export const priorityTierEnum = pgEnum("priorityTier", [
  "critical",
  "high",
  "medium",
  "low",
]);

export const runStatusEnum = pgEnum("runStatus", [
  "queued",
  "running",
  "completed",
  "failed",
]);

export const triggeredByEnum = pgEnum("triggeredBy", [
  "manual",
  "scheduled",
  "webhook",
]);

export const severityImpactEnum = pgEnum("severityImpact", [
  "critical",
  "high",

```

```

    "medium",
    "low",
  ]);

export const onboardingStageEnum = pgEnum("onboardingStage", [
  "not_started",
  "account_type_selected",
  "org_created",
  "org_joined",
  "jurisdiction_set",
  "completed",
]);

export const accountIntentEnum = pgEnum("accountIntent", [
  "compliance_professional",
  "legal_advisor",
  "enterprise_admin",
  "consultant",
  "vendor",
  "government",
  "researcher",
]);

export const evidenceSourceTypeEnum = pgEnum("evidenceSourceType", [
  "audit_schedule",
  "policy",
  "risk",
  "gap",
  "remediation",
  "ctem_asset",
  "incident",
  "general",
]);

export const dsrRequestTypeEnum = pgEnum("dsrRequestType", [
  "access",
  "rectification",
  "erasure",
  "portability",
  "restriction",
  "objection",
  "explanation",
]);

export const dsrJurisdictionEnum = pgEnum("dsrJurisdiction", [
  "China",
  "Saudi Arabia",
  "Other",
]);

export const dsrStatusEnum = pgEnum("dsrStatus", [
  "received",
  "in_review",
  "pending_info",
  "completed",
  "rejected",
  "withdrawn",
]);

export const dsrPriorityEnum = pgEnum("dsrPriority", [
  "normal",
  "high",
  "urgent",
]);

export const serviceTypeEnum = pgEnum("serviceType", [
  "penetration_test",
  "red_team",
  "security_audit",
  "soc_support",
  "incident_response",
  "consulting",
  "phishing_simulation",
  "cloud_security_review",
]);

```

```

    "vulnerability_assessment",
    "compliance_gap_assessment",
  ]);

export const servicePriorityEnum = pgEnum("servicePriority", [
  "low",
  "medium",
  "high",
  "critical",
]);

export const serviceStatusEnum = pgEnum("serviceStatus", [
  "draft",
  "submitted",
  "under_review",
  "scoping",
  "approved",
  "in_progress",
  "completed",
  "cancelled",
  "on_hold",
]);

export const inventoryAssetTypeEnum = pgEnum("inventoryAssetType", [
  "server",
  "workstation",
  "network_device",
  "cloud_service",
  "saas_application",
  "database",
  "api_endpoint",
  "iot_device",
  "mobile_device",
  "industrial_ot",
  "web_application",
  "source_code_repo",
  "third_party_service",
]);

export const exposureEnum = pgEnum("exposure", [
  "internal",
  "vpn_only",
  "partner_only",
  "internet_facing",
]);

export const inventoryStatusEnum = pgEnum("inventoryStatus", [
  "active",
  "decommissioned",
  "under_review",
  "unknown",
]);

export const maturityLevelEnum = pgEnum("maturityLevel", [
  "initial",
  "developing",
  "defined",
  "managed",
  "optimized",
]);

export const threatCategoryEnum = pgEnum("threatCategory", [
  "malware",
  "ransomware",
  "phishing",
  "apt",
  "zero_day",
  "ddos",
  "supply_chain",
  "data_breach",
  "vulnerability",
  "social_engineering",
  "insider_threat",
  "other",
]);

```



```

});

export const threatSeverityEnum = pgEnum("threatSeverity", [
  "info",
  "low",
  "medium",
  "high",
  "critical",
]);

export const tlpEnum = pgEnum("tlp", ["white", "green", "amber", "red"]);

export const deadlineJurisdictionEnum = pgEnum("deadlineJurisdiction", [
  "China",
  "Saudi Arabia",
  "Both",
]);

export const deadlinePriorityEnum = pgEnum("deadlinePriority", [
  "low",
  "medium",
  "high",
  "critical",
]);

export const deadlineStatusEnum = pgEnum("deadlineStatus", [
  "upcoming",
  "overdue",
  "completed",
  "waived",
]);

//
// TABLES
//

// Local Auth Users
export const localUsers = pgTable("localUsers", {
  id: serial("id").primaryKey(),
  name: varchar("name", { length: 255 }).notNull(),
  email: varchar("email", { length: 320 }).unique(),
  phoneNumber: varchar("phoneNumber", { length: 20 }).unique(),
  passwordHash: varchar("passwordHash", { length: 72 }),
  userType: userTypeEnum("userType").default("visitor").notNull(),
  companyName: varchar("companyName", { length: 255 }),
  jobTitle: varchar("jobTitle", { length: 120 }),
  industry: varchar("industry", { length: 120 }),
  complianceResponsibility: text("complianceResponsibility"),
  preferredLocale: localeEnum("preferredLocale").default("en").notNull(),
  status: userStatusEnum("status").default("pending").notNull(),
  lastSignedIn: timestamp("lastSignedIn"),
  totpSecret: varchar("totpSecret", { length: 64 }),
  mfaEnabled: integer("mfaEnabled").default(0).notNull(),
  mfaBackupCodes: text("mfaBackupCodes"),
  verifiedAt: timestamp("verifiedAt"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type LocalUser = typeof localUsers.$inferSelect;
export type InsertLocalUser = typeof localUsers.$inferInsert;

export const users = pgTable("users", {
  id: serial("id").primaryKey(),
  openId: varchar("openId", { length: 64 }).notNull().unique(),
  name: text("name"),
  email: varchar("email", { length: 320 }),
  loginMethod: varchar("loginMethod", { length: 64 }),
  organizationName: varchar("organizationName", { length: 255 }),
  organizationType: varchar("organizationType", { length: 120 }),
  jobTitle: varchar("jobTitle", { length: 120 }),
  preferredLocale: localeEnum("preferredLocale").default("en").notNull(),
  role: roleEnum("role").default("user").notNull(),
  status: userOAuthStatusEnum("status").default("active").notNull(),
});

```

```

    createdAt: timestamp("createdAt").defaultNow().notNull(),
    updatedAt: timestamp("updatedAt").defaultNow().notNull(),
    lastSignedIn: timestamp("lastSignedIn").defaultNow().notNull(),
    lastActivityAt: timestamp("lastActivityAt").defaultNow().notNull(),
  });

export type User = typeof users.$inferSelect;
export type InsertUser = typeof users.$inferInsert;

export type AllTables = {
  localUsers: typeof localUsers;
  users: typeof users;
  accessRequests: typeof accessRequests;
  consultationRequests: typeof consultationRequests;
  activityEvents: typeof activityEvents;
  adminNotifications: typeof adminNotifications;
  frameworks: typeof frameworks;
  complianceControls: typeof complianceControls;
  frameworkRelationships: typeof frameworkRelationships;
  controlMappings: typeof controlMappings;
  vendors: typeof vendors;
  techStackComponents: typeof techStackComponents;
  vendorAssessments: typeof vendorAssessments;
  assessmentGaps: typeof assessmentGaps;
  auditLogs: typeof auditLogs;
};

export const accessRequests = pgTable("accessRequests", {
  id: serial("id").primaryKey(),
  fullName: varchar("fullName", { length: 255 }).notNull(),
  email: varchar("email", { length: 320 }).notNull(),
  organizationName: varchar("organizationName", { length: 255 }).notNull(),
  organizationType: varchar("organizationType", { length: 120 }),
  useCase: text("useCase"),
  preferredLocale: localeEnum("preferredLocale").default("en").notNull(),
  status: accessRequestStatusEnum("status").default("new").notNull(),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type AccessRequest = typeof accessRequests.$inferSelect;
export type InsertAccessRequest = typeof accessRequests.$inferInsert;

export const consultationRequests = pgTable("consultationRequests", {
  id: serial("id").primaryKey(),
  userId: integer("userId").references(() => users.id),
  contactName: varchar("contactName", { length: 255 }).notNull(),
  contactEmail: varchar("contactEmail", { length: 320 }).notNull(),
  organizationName: varchar("organizationName", { length: 255 }).notNull(),
  topic: varchar("topic", { length: 255 }).notNull(),
  jurisdictions: text("jurisdictions"),
  summary: text("summary").notNull(),
  vendorName: varchar("vendorName", { length: 255 }),
  techStackSummary: text("techStackSummary"),
  status: consultationStatusEnum("status").default("new").notNull(),
  priority: priorityEnum("priority").default("medium").notNull(),
  assignedAdminUserId: integer("assignedAdminUserId").references(
    () => users.id
  ),
  adminResponse: text("adminResponse"),
  respondedAt: timestamp("respondedAt"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type ConsultationRequest = typeof consultationRequests.$inferSelect;
export type InsertConsultationRequest =
  typeof consultationRequests.$inferInsert;

export const activityEvents = pgTable("activityEvents", {
  id: serial("id").primaryKey(),
  userId: integer("userId").references(() => users.id),
  localUserId: integer("localUserId").references(() => localUsers.id),
  actorType: actorTypeEnum("actorType").notNull(),

```

```

    actorRole: varchar("actorRole", { length: 64 }),
    action: varchar("action", { length: 120 }).notNull(),
    entityType: varchar("entityType", { length: 120 }).notNull(),
    entityId: integer("entityId"),
    targetEntity: varchar("targetEntity", { length: 255 }),
    payload: text("payload"),
    ipHash: varchar("ipHash", { length: 64 }),
    metadata: text("metadata"),
    createdAt: timestamp("createdAt").defaultNow().notNull(),
  });

export type ActivityEvent = typeof activityEvents.$inferSelect;
export type InsertActivityEvent = typeof activityEvents.$inferInsert;

export const adminNotifications = pgTable("adminNotifications", {
  id: serial("id").primaryKey(),
  category: notificationCategoryEnum("category").notNull(),
  title: varchar("title", { length: 255 }).notNull(),
  content: text("content"),
  entityType: varchar("entityType", { length: 120 }),
  entityId: integer("entityId"),
  isRead: integer("isRead").default(0).notNull(),
  readAt: timestamp("readAt"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});

export type AdminNotification = typeof adminNotifications.$inferSelect;
export type InsertAdminNotification = typeof adminNotifications.$inferInsert;

export const frameworks = pgTable("frameworks", {
  id: serial("id").primaryKey(),
  code: varchar("code", { length: 50 }).notNull().unique(),
  name: text("name").notNull(),
  country: varchar("country", { length: 50 }).notNull(),
  description: text("description"),
  scope: text("scope"),
  enforcementAuthority: varchar("enforcementAuthority", { length: 255 }),
  maxPenalty: varchar("maxPenalty", { length: 255 }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type Framework = typeof frameworks.$inferSelect;
export type InsertFramework = typeof frameworks.$inferInsert;

export const complianceControls = pgTable("complianceControls", {
  id: serial("id").primaryKey(),
  frameworkId: integer("frameworkId")
    .notNull()
    .references(() => frameworks.id),
  controlCode: varchar("controlCode", { length: 100 }).notNull(),
  controlName: text("controlName").notNull(),
  category: varchar("category", { length: 100 }),
  description: text("description"),
  requirement: text("requirement"),
  applicability: varchar("applicability", { length: 255 }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type ComplianceControl = typeof complianceControls.$inferSelect;
export type InsertComplianceControl = typeof complianceControls.$inferInsert;

export const frameworkRelationships = pgTable("frameworkRelationships", {
  id: serial("id").primaryKey(),
  sourceFrameworkId: integer("sourceFrameworkId")
    .notNull()
    .references(() => frameworks.id),
  targetFrameworkId: integer("targetFrameworkId")
    .notNull()
    .references(() => frameworks.id),
  relationshipType: relationshipTypeEnum("relationshipType").notNull(),
  description: text("description"),
  severity: severityEnum("severity"),
});

```

```

    riskLevel: varchar("riskLevel", { length: 50 }),
    mitigation: text("mitigation"),
    createdAt: timestamp("createdAt").defaultNow().notNull(),
    updatedAt: timestamp("updatedAt").defaultNow().notNull(),
  });

export type FrameworkRelationship = typeof frameworkRelationships.$inferSelect;
export type InsertFrameworkRelationship =
  typeof frameworkRelationships.$inferInsert;

export const controlMappings = pgTable("controlMappings", {
  id: serial("id").primaryKey(),
  sourceControlId: integer("sourceControlId")
    .notNull()
    .references(() => complianceControls.id),
  targetControlId: integer("targetControlId")
    .notNull()
    .references(() => complianceControls.id),
  mappingType: mappingTypeEnum("mappingType").notNull(),
  alignmentScore: integer("alignmentScore"),
  notes: text("notes"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type ControlMapping = typeof controlMappings.$inferSelect;
export type InsertControlMapping = typeof controlMappings.$inferInsert;

export const vendors = pgTable("vendors", {
  id: serial("id").primaryKey(),
  userId: integer("userId")
    .notNull()
    .references(() => users.id),
  organizationId: integer("organizationId").references(() => organizations.id),
  vendorName: varchar("vendorName", { length: 255 }).notNull(),
  vendorDescription: text("vendorDescription"),
  industry: varchar("industry", { length: 100 }),
  businessRegistrationNumber: varchar("businessRegistrationNumber", {
    length: 120,
  }),
  headquartersLocation: varchar("headquartersLocation", { length: 120 }),
  primaryContactName: varchar("primaryContactName", { length: 255 }),
  primaryContactEmail: varchar("primaryContactEmail", { length: 320 }),
  primaryContactRole: varchar("primaryContactRole", { length: 120 }),
  primaryContactPhone: varchar("primaryContactPhone", { length: 64 }),
  serviceType: varchar("serviceType", { length: 120 }),
  serviceScope: text("serviceScope"),
  hostingEnvironment: varchar("hostingEnvironment", { length: 120 }),
  operatingCountries: text("operatingCountries"),
  cloudProvider: varchar("cloudProvider", { length: 255 }),
  dataLocations: text("dataLocations"),
  regulatoryJurisdictions: text("regulatoryJurisdictions"),
  certifications: text("certifications"),
  dataProcessingActivities: text("dataProcessingActivities"),
  criticalityLevel: varchar("criticalityLevel", { length: 64 }),
  riskTier: varchar("riskTier", { length: 64 }),
  thirdPartyDependencies: varchar("thirdPartyDependencies", { length: 64 }),
  fourthPartyDependencies: varchar("fourthPartyDependencies", { length: 64 }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type Vendor = typeof vendors.$inferSelect;
export type InsertVendor = typeof vendors.$inferInsert;

export const techStackComponents = pgTable("techStackComponents", {
  id: serial("id").primaryKey(),
  vendorId: integer("vendorId")
    .notNull()
    .references(() => vendors.id),
  componentName: varchar("componentName", { length: 255 }).notNull(),
  componentType: varchar("componentType", { length: 100 }),
  technology: varchar("technology", { length: 255 }),
  description: text("description"),

```

```

    dataHandling: text("dataHandling"),
    createdAt: timestamp("createdAt").defaultNow().notNull(),
    updatedAt: timestamp("updatedAt").defaultNow().notNull(),
  });

export type TechStackComponent = typeof techStackComponents.$inferSelect;
export type InsertTechStackComponent = typeof techStackComponents.$inferInsert;

export const vendorAssessments = pgTable("vendorAssessments", {
  id: serial("id").primaryKey(),
  vendorId: integer("vendorId")
    .notNull()
    .references(() => vendors.id),
  frameworkId: integer("frameworkId")
    .notNull()
    .references(() => frameworks.id),
  assessmentDate: timestamp("assessmentDate").defaultNow().notNull(),
  complianceScore: integer("complianceScore"),
  riskLevel: severityEnum("riskLevel"),
  status: assessmentStatusEnum("status"),
  findings: text("findings"),
  recommendations: text("recommendations"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type VendorAssessment = typeof vendorAssessments.$inferSelect;
export type InsertVendorAssessment = typeof vendorAssessments.$inferInsert;

export const assessmentGaps = pgTable("assessmentGaps", {
  id: serial("id").primaryKey(),
  assessmentId: integer("assessmentId")
    .notNull()
    .references(() => vendorAssessments.id),
  controlId: integer("controlId")
    .notNull()
    .references(() => complianceControls.id),
  gapDescription: text("gapDescription"),
  severity: severityEnum("severity"),
  remediation: text("remediation"),
  estimatedRemediationCost: varchar("estimatedRemediationCost", {
    length: 100,
  }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type AssessmentGap = typeof assessmentGaps.$inferSelect;
export type InsertAssessmentGap = typeof assessmentGaps.$inferInsert;

//
// SAAS MULTI-TENANCY & BILLING
//

export const organizations = pgTable("organizations", {
  id: serial("id").primaryKey(),
  slug: varchar("slug", { length: 100 }).notNull().unique(),
  name: varchar("name", { length: 255 }).notNull(),
  billingEmail: varchar("billingEmail", { length: 320 }).notNull(),
  industry: varchar("industry", { length: 120 }),
  primaryJurisdiction: jurisdictionEnum("primaryJurisdiction").default("Both"),
  stripeCustomerId: varchar("stripeCustomerId", { length: 64 }),
  plan: planEnum("plan").default("free_trial").notNull(),
  trialStartedAt: timestamp("trialStartedAt"),
  trialEndsAt: timestamp("trialEndsAt"),
  trialReminderDay3Sent: integer("trialReminderDay3Sent").default(0).notNull(),
  trialReminderDay6Sent: integer("trialReminderDay6Sent").default(0).notNull(),
  trialExpiredNoticeSent: integer("trialExpiredNoticeSent")
    .default(0)
    .notNull(),
  isActive: integer("isActive").default(1).notNull(),
  maxSeats: integer("maxSeats").default(5).notNull(),
  metadata: text("metadata"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});

```

```

    updatedAt: timestamp("updatedAt").defaultNow().notNull(),
  });

export type Organization = typeof organizations.$inferSelect;
export type InsertOrganization = typeof organizations.$inferInsert;

export const organizationMembers = pgTable("organizationMembers", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id),
  userId: integer("userId").references(() => users.id),
  localUserId: integer("localUserId").references(() => localUsers.id),
  role: orgMemberRoleEnum("role").default("analyst").notNull(),
  invitedByUserId: integer("invitedByUserId"),
  inviteEmail: varchar("inviteEmail", { length: 320 }),
  inviteToken: varchar("inviteToken", { length: 64 }),
  inviteAcceptedAt: timestamp("inviteAcceptedAt"),
  status: orgMemberStatusEnum("status").default("active").notNull(),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type OrganizationMember = typeof organizationMembers.$inferSelect;
export type InsertOrganizationMember = typeof organizationMembers.$inferInsert;

export const subscriptions = pgTable("subscriptions", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id),
  stripeSubscriptionId: varchar("stripeSubscriptionId", {
    length: 64,
  }).unique(),
  stripePriceId: varchar("stripePriceId", { length: 64 }),
  plan: paidPlanEnum("plan").notNull(),
  billingInterval: billingIntervalEnum("billingInterval").notNull(),
  amountCents: integer("amountCents").notNull(),
  currency: varchar("currency", { length: 3 }).default("USD").notNull(),
  status: subscriptionStatusEnum("status").default("trialing").notNull(),
  currentPeriodStart: timestamp("currentPeriodStart"),
  currentPeriodEnd: timestamp("currentPeriodEnd"),
  cancelAtPeriodEnd: integer("cancelAtPeriodEnd").default(0).notNull(),
  canceledAt: timestamp("canceledAt"),
  lastInvoiceId: varchar("lastInvoiceId", { length: 64 }),
  stripeMetadata: text("stripeMetadata"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type Subscription = typeof subscriptions.$inferSelect;
export type InsertSubscription = typeof subscriptions.$inferInsert;

export const billingEvents = pgTable("billingEvents", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id),
  subscriptionId: integer("subscriptionId").references(() => subscriptions.id),
  stripeEventId: varchar("stripeEventId", { length: 64 }).unique(),
  eventType: varchar("eventType", { length: 120 }).notNull(),
  status: billingEventStatusEnum("status").default("pending").notNull(),
  amountCents: integer("amountCents"),
  currency: varchar("currency", { length: 3 }).default("USD"),
  description: text("description"),
  rawPayload: text("rawPayload"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});

export type BillingEvent = typeof billingEvents.$inferSelect;
export type InsertBillingEvent = typeof billingEvents.$inferInsert;

export const complianceReports = pgTable("complianceReports", {
  id: serial("id").primaryKey(),

```

```

organizationId: integer("organizationId")
  .notNull()
  .references(() => organizations.id),
generatedByUserId: integer("generatedByUserId").references(() => users.id),
generatedByLocalUserId: integer("generatedByLocalUserId").references(
  () => localUsers.id
),
title: varchar("title", { length: 255 }).notNull(),
reportType: reportTypeEnum("reportType").notNull(),
frameworks: text("frameworks").notNull(),
aiJobId: varchar("aiJobId", { length: 64 }),
version: integer("version").default(1).notNull(),
overallScore: integer("overallScore"),
riskLevel: severityEnum("riskLevel"),
reportBody: text("reportBody").notNull(),
exportedAt: timestamp("exportedAt"),
exportedPdfUrl: varchar("exportedPdfUrl", { length: 512 }),
status: reportStatusEnum("status").default("generating").notNull(),
createdAt: timestamp("createdAt").defaultNow().notNull(),
updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type ComplianceReport = typeof complianceReports.$inferSelect;
export type InsertComplianceReport = typeof complianceReports.$inferInsert;

export const apiKeys = pgTable("apiKeys", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id, { onDelete: "cascade" }),
  createdByUserId: integer("createdByUserId").references(() => users.id),
  name: varchar("name", { length: 120 }).notNull(),
  keyHash: varchar("keyHash", { length: 64 }).notNull().unique(),
  keyPrefix: varchar("keyPrefix", { length: 16 }).notNull(),
  scopes: text("scopes"),
  lastUsedAt: timestamp("lastUsedAt"),
  expiresAt: timestamp("expiresAt"),
  revokedAt: timestamp("revokedAt"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});

export type ApiKey = typeof apiKeys.$inferSelect;
export type InsertApiKey = typeof apiKeys.$inferInsert;

export const userInteractionLogs = pgTable("userInteractionLogs", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId").references(() => organizations.id),
  userId: integer("userId").references(() => users.id),
  localUserId: integer("localUserId").references(() => localUsers.id),
  sessionId: varchar("sessionId", { length: 64 }),
  context: varchar("context", { length: 120 }).notNull(),
  action: varchar("action", { length: 120 }).notNull(),
  entityType: varchar("entityType", { length: 120 }),
  entityId: integer("entityId"),
  inputSnapshot: text("inputSnapshot"),
  outputRef: text("outputRef"),
  durationMs: integer("durationMs"),
  ipHash: varchar("ipHash", { length: 64 }),
  userAgent: varchar("userAgent", { length: 512 }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});

export type UserInteractionLog = typeof userInteractionLogs.$inferSelect;
export type InsertUserInteractionLog = typeof userInteractionLogs.$inferInsert;

export const yallaAdminAccessLinkNonces = pgTable(
  "yallaAdminAccessLinkNonces",
  {
    id: serial("id").primaryKey(),
    nonceHash: varchar("nonceHash", { length: 64 }).notNull().unique(),
    redirectTarget: varchar("redirectTarget", { length: 255 }).notNull(),
    expiresAt: timestamp("expiresAt").notNull(),
    consumedAt: timestamp("consumedAt").defaultNow().notNull(),
    consumedByIp: varchar("consumedByIp", { length: 64 }).notNull(),
  }
);

```

```

    createdAt: timestamp("createdAt").defaultNow().notNull(),
  }
};

export type YallaAdminAccessLinkNonce =
  typeof yallaAdminAccessLinkNonces.$inferSelect;
export type InsertYallaAdminAccessLinkNonce =
  typeof yallaAdminAccessLinkNonces.$inferInsert;

export const complianceDeadlines = pgTable("complianceDeadlines", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId").references(() => organizations.id),
  frameworkCode: varchar("frameworkCode", { length: 50 }).notNull(),
  title: varchar("title", { length: 255 }).notNull(),
  description: text("description"),
  deadlineDate: timestamp("deadlineDate").notNull(),
  jurisdiction: deadlineJurisdictionEnum("jurisdiction").notNull(),
  priority: deadlinePriorityEnum("priority").default("medium").notNull(),
  status: deadlineStatusEnum("status").default("upcoming").notNull(),
  notificationsSent: text("notificationsSent"),
  assignedToUserId: integer("assignedToUserId").references(() => users.id),
  completedAt: timestamp("completedAt"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type ComplianceDeadline = typeof complianceDeadlines.$inferSelect;
export type InsertComplianceDeadline = typeof complianceDeadlines.$inferInsert;

export const auditLogs = pgTable("auditLogs", {
  id: serial("id").primaryKey(),
  userId: integer("userId").references(() => users.id),
  localUserId: integer("localUserId").references(() => localUsers.id),
  organizationId: integer("organizationId").references(() => organizations.id),
  actorRole: varchar("actorRole", { length: 64 }),
  category: auditLogCategoryEnum("category").notNull(),
  action: varchar("action", { length: 120 }).notNull(),
  entityType: varchar("entityType", { length: 120 }),
  entityId: integer("entityId"),
  targetEntity: varchar("targetEntity", { length: 255 }),
  outcome: auditLogOutcomeEnum("outcome").default("success").notNull(),
  payload: text("payload"),
  ipHash: varchar("ipHash", { length: 64 }),
  userAgent: varchar("userAgent", { length: 512 }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});

export type AuditLog = typeof auditLogs.$inferSelect;
export type InsertAuditLog = typeof auditLogs.$inferInsert;

// Report Shares
export const reportShares = pgTable("reportShares", {
  id: serial("id").primaryKey(),
  token: varchar("token", { length: 64 }).notNull().unique(),
  jurisdiction: varchar("jurisdiction", { length: 64 }).notNull(),
  locale: localeEnum("locale").default("en").notNull(),
  reportType: varchar("reportType", { length: 64 }).notNull(),
  createdByUserId: integer("createdByUserId").references(() => users.id),
  viewCount: integer("viewCount").default(0).notNull(),
  expiresAt: timestamp("expiresAt").notNull(),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});

export type ReportShare = typeof reportShares.$inferSelect;
export type InsertReportShare = typeof reportShares.$inferInsert;

// Remediation Tasks
export const remediationTasks = pgTable("remediationTasks", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id, { onDelete: "cascade" }),
  vendorId: integer("vendorId"),
  gapCode: varchar("gapCode", { length: 64 }),

```



```

    title: varchar("title", { length: 255 }).notNull(),
    description: text("description"),
    severity: taskSeverityEnum("severity").notNull().default("medium"),
    status: taskStatusEnum("status").notNull().default("open"),
    assignedToUserId: integer("assignedToUserId"),
    dueDate: timestamp("dueDate"),
    notes: text("notes"),
    createdAt: timestamp("createdAt").defaultNow().notNull(),
    updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type RemediationTask = typeof remediationTasks.$inferSelect;
export type InsertRemediationTask = typeof remediationTasks.$inferInsert;

// Risk Register
export const riskRegister = pgTable("riskRegister", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id, { onDelete: "cascade" }),
  title: varchar("title", { length: 255 }).notNull(),
  description: text("description"),
  category: riskCategoryEnum("category").notNull().default("operational"),
  likelihood: integer("likelihood").notNull().default(3),
  impact: integer("impact").notNull().default(3),
  treatment: treatmentEnum("treatment").notNull().default("mitigate"),
  status: riskStatusEnum("status").notNull().default("open"),
  ownerId: integer("ownerId"),
  vendorId: integer("vendorId"),
  gapCode: varchar("gapCode", { length: 64 }),
  controlReference: varchar("controlReference", { length: 128 }),
  reviewDate: timestamp("reviewDate"),
  notes: text("notes"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type RiskEntry = typeof riskRegister.$inferSelect;
export type InsertRiskEntry = typeof riskRegister.$inferInsert;

// Compliance Policies
export const compliancePolicies = pgTable("compliancePolicies", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id, { onDelete: "cascade" }),
  policyCode: varchar("policyCode", { length: 64 }),
  title: varchar("title", { length: 255 }).notNull(),
  description: text("description"),
  policyType: policyTypeEnum("policyType").notNull().default("policy"),
  frameworks: text("frameworks"),
  controlReferences: text("controlReferences"),
  status: policyStatusEnum("status").notNull().default("draft"),
  ownerId: integer("ownerId"),
  reviewCycleMonths: integer("reviewCycleMonths").default(12),
  lastApprovedAt: timestamp("lastApprovedAt"),
  nextReviewAt: timestamp("nextReviewAt"),
  version: varchar("version", { length: 20 }).default("1.0"),
  documentUrl: varchar("documentUrl", { length: 512 }),
  notes: text("notes"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type CompliancePolicy = typeof compliancePolicies.$inferSelect;
export type InsertCompliancePolicy = typeof compliancePolicies.$inferInsert;

// Compliance Incident Register
export const complianceIncidents = pgTable("complianceIncidents", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id, { onDelete: "cascade" }),
  incidentCode: varchar("incidentCode", { length: 64 }),

```

```

title: varchar("title", { length: 255 }).notNull(),
description: text("description"),
incidentType: incidentTypeEnum("incidentType").default("other").notNull(),
severity: taskSeverityEnum("severity").default("medium").notNull(),
status: incidentStatusEnum("status").default("open").notNull(),
affectedFrameworks: text("affectedFrameworks"),
affectedVendorId: integer("affectedVendorId"),
affectedDataTypes: text("affectedDataTypes"),
affectedDataSubjects: integer("affectedDataSubjects"),
reportedById: integer("reportedById"),
occurredAt: timestamp("occurredAt"),
detectedAt: timestamp("detectedAt"),
containedAt: timestamp("containedAt"),
resolvedAt: timestamp("resolvedAt"),
regulatoryNotificationRequired: integer("regulatoryNotificationRequired")
  .default(0)
  .notNull(),
regulatoryNotificationSentAt: timestamp("regulatoryNotificationSentAt"),
notificationDeadlineHours: integer("notificationDeadlineHours").default(72),
rootCause: text("rootCause"),
lessonsLearned: text("lessonsLearned"),
notes: text("notes"),
createdAt: timestamp("createdAt").defaultNow().notNull(),
updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type ComplianceIncident = typeof complianceIncidents.$inferSelect;
export type InsertComplianceIncident = typeof complianceIncidents.$inferInsert;

// Audit Schedules
export const auditSchedules = pgTable("auditSchedules", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id, { onDelete: "cascade" }),
  title: varchar("title", { length: 255 }).notNull(),
  description: text("description"),
  auditType: auditTypeEnum("auditType").default("internal").notNull(),
  scope: text("scope"),
  status: auditStatusEnum("status").default("planned").notNull(),
  scheduledDate: timestamp("scheduledDate").notNull(),
  completedDate: timestamp("completedDate"),
  assignedToId: integer("assignedToId"),
  vendorId: integer("vendorId"),
  findings: text("findings"),
  recurrence: recurrenceEnum("recurrence").default("none").notNull(),
  nextOccurrence: timestamp("nextOccurrence"),
  notes: text("notes"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type AuditSchedule = typeof auditSchedules.$inferSelect;
export type InsertAuditSchedule = typeof auditSchedules.$inferInsert;

//
// CTEM Continuous Threat Exposure Management
//

export const ctemAssets = pgTable("ctemAssets", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id, { onDelete: "cascade" }),
  vendorId: integer("vendorId").references(() => vendors.id, {
    onDelete: "set null",
  }),
  assetName: varchar("assetName", { length: 255 }).notNull(),
  assetType: ctemAssetTypeEnum("assetType").notNull().default("other"),
  ipDomain: varchar("ipDomain", { length: 255 }),
  region: regionEnum("region").notNull().default("Other"),
  isInternetFacing: integer("isInternetFacing").default(0).notNull(),
  handlesPersonalData: integer("handlesPersonalData").default(0).notNull(),
  handlesCriticalData: integer("handlesCriticalData").default(0).notNull(),
});

```

```

    criticalityScore: integer("criticalityScore").default(5).notNull(),
    status: assetStatusEnum("status").default("active").notNull(),
    notes: text("notes"),
    createdAt: timestamp("createdAt").defaultNow().notNull(),
    updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type CtemAsset = typeof ctemAssets.$inferSelect;
export type InsertCtemAsset = typeof ctemAssets.$inferInsert;

export const ctemVulnerabilities = pgTable("ctemVulnerabilities", {
  id: serial("id").primaryKey(),
  assetId: integer("assetId")
    .notNull()
    .references(() => ctemAssets.id, { onDelete: "cascade" }),
  cveId: varchar("cveId", { length: 64 }),
  title: varchar("title", { length: 255 }).notNull(),
  description: text("description"),
  severity: vulnSeverityEnum("severity").notNull().default("medium"),
  cvssScore: integer("cvssScore").default(0).notNull(),
  exploitAvailable: integer("exploitAvailable").default(0).notNull(),
  isConfirmed: integer("isConfirmed").default(0).notNull(),
  isPatched: integer("isPatched").default(0).notNull(),
  discoveredAt: timestamp("discoveredAt").defaultNow().notNull(),
  patchedAt: timestamp("patchedAt"),
  notes: text("notes"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type CtemVulnerability = typeof ctemVulnerabilities.$inferSelect;
export type InsertCtemVulnerability = typeof ctemVulnerabilities.$inferInsert;

export const ctemAttackSimulations = pgTable("ctemAttackSimulations", {
  id: serial("id").primaryKey(),
  assetId: integer("assetId")
    .notNull()
    .references(() => ctemAssets.id, { onDelete: "cascade" }),
  simulationType: simulationTypeEnum("simulationType")
    .notNull()
    .default("other"),
  successProbability: integer("successProbability").default(0).notNull(),
  executedAt: timestamp("executedAt").defaultNow().notNull(),
  outputSummary: text("outputSummary"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});

export type CtemAttackSimulation = typeof ctemAttackSimulations.$inferSelect;
export type InsertCtemAttackSimulation =
  typeof ctemAttackSimulations.$inferInsert;

export const ctemRiskScores = pgTable("ctemRiskScores", {
  id: serial("id").primaryKey(),
  assetId: integer("assetId")
    .notNull()
    .references(() => ctemAssets.id, { onDelete: "cascade" }),
  exposureScore: integer("exposureScore").default(0).notNull(),
  exploitabilityScore: integer("exploitabilityScore").default(0).notNull(),
  businessImpactScore: integer("businessImpactScore").default(0).notNull(),
  finalPriorityScore: integer("finalPriorityScore").default(0).notNull(),
  priorityTier: priorityTierEnum("priorityTier").notNull().default("low"),
  previousFinalScore: integer("previousFinalScore"),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type CtemRiskScore = typeof ctemRiskScores.$inferSelect;
export type InsertCtemRiskScore = typeof ctemRiskScores.$inferInsert;

export const continuousComplianceRuns = pgTable("continuousComplianceRuns", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id, { onDelete: "cascade" }),
  vendorId: integer("vendorId").references(() => vendors.id, {

```

```

    onDelete: "set null",
  }},
  runStatus: runStatusEnum("runStatus").notNull().default("queued"),
  triggeredBy: triggeredByEnum("triggeredBy").notNull().default("manual"),
  aiJobId: varchar("aiJobId", { length: 64 }),
  assetsScanned: integer("assetsScanned").default(0).notNull(),
  vulnsFound: integer("vulnsFound").default(0).notNull(),
  exploitableVulns: integer("exploitableVulns").default(0).notNull(),
  avgPriorityScore: integer("avgPriorityScore").default(0).notNull(),
  scoreDelta: integer("scoreDelta"),
  alertRaised: integer("alertRaised").default(0).notNull(),
  summary: text("summary"),
  startedAt: timestamp("startedAt").defaultNow().notNull(),
  completedAt: timestamp("completedAt"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});

export type ContinuousComplianceRun =
  typeof continuousComplianceRuns.$inferSelect;
export type InsertContinuousComplianceRun =
  typeof continuousComplianceRuns.$inferInsert;

export const complianceExposureMappings = pgTable(
  "complianceExposureMappings",
  {
    id: serial("id").primaryKey(),
    vulnerabilityId: integer("vulnerabilityId")
      .notNull()
      .references(() => ctemVulnerabilities.id, { onDelete: "cascade" }),
    frameworkId: integer("frameworkId").references(() => frameworks.id, {
      onDelete: "set null",
    }),
    frameworkCode: varchar("frameworkCode", { length: 50 }),
    controlId: integer("controlId").references(() => complianceControls.id, {
      onDelete: "set null",
    }),
    controlCode: varchar("controlCode", { length: 100 }),
    mappingReason: text("mappingReason").notNull(),
    severityImpact: severityImpactEnum("severityImpact")
      .notNull()
      .default("medium"),
    createdAt: timestamp("createdAt").defaultNow().notNull(),
  }
);

export type ComplianceExposureMapping =
  typeof complianceExposureMappings.$inferSelect;
export type InsertComplianceExposureMapping =
  typeof complianceExposureMappings.$inferInsert;

//
// RBAC Role-Based Access Control & Onboarding
//

export const userOnboarding = pgTable("userOnboarding", {
  id: serial("id").primaryKey(),
  userId: integer("userId").references(() => users.id, { onDelete: "cascade" }),
  localUserId: integer("localUserId").references(() => localUsers.id, {
    onDelete: "cascade",
  }),
  stage: onboardingStageEnum("stage").default("not_started").notNull(),
  accountIntent: accountIntentEnum("accountIntent"),
  selectedLocale: localeEnum("selectedLocale").default("en").notNull(),
  completedAt: timestamp("completedAt"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type UserOnboarding = typeof userOnboarding.$inferSelect;
export type InsertUserOnboarding = typeof userOnboarding.$inferInsert;

export const rolePermissions = pgTable("rolePermissions", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")

```

```

        .NotNull()
        .references(() => organizations.id, { onDelete: "cascade" })),
    userId: integer("userId").references(() => users.id, { onDelete: "cascade" })),
    localUserId: integer("localUserId").references(() => localUsers.id, {
        onDelete: "cascade",
    })),
    module: varchar("module", { length: 120 }).NotNull(),
    canView: integer("canView").default(0).NotNull(),
    canCreate: integer("canCreate").default(0).NotNull(),
    canEdit: integer("canEdit").default(0).NotNull(),
    canDelete: integer("canDelete").default(0).NotNull(),
    canExport: integer("canExport").default(0).NotNull(),
    canInvite: integer("canInvite").default(0).NotNull(),
    grantedByUserId: integer("grantedByUserId").references(() => users.id),
    createdAt: timestamp("createdAt").defaultNow().NotNull(),
    updatedAt: timestamp("updatedAt").defaultNow().NotNull(),
});

export type RolePermission = typeof rolePermissions.$inferSelect;
export type InsertRolePermission = typeof rolePermissions.$inferInsert;

export const vendorShares = pgTable("vendorShares", {
    id: serial("id").primaryKey(),
    vendorId: integer("vendorId")
        .NotNull()
        .references(() => vendors.id, { onDelete: "cascade" })),
    shareToken: varchar("shareToken", { length: 64 }).NotNull().unique(),
    allowedOrgId: integer("allowedOrgId").references(() => organizations.id, {
        onDelete: "set null",
    })),
    expiresAt: timestamp("expiresAt"),
    viewCount: integer("viewCount").default(0).NotNull(),
    createdByUserId: integer("createdByUserId").references(() => users.id),
    isRevoked: integer("isRevoked").default(0).NotNull(),
    revokedAt: timestamp("revokedAt"),
    createdAt: timestamp("createdAt").defaultNow().NotNull(),
});

export type VendorShare = typeof vendorShares.$inferSelect;
export type InsertVendorShare = typeof vendorShares.$inferInsert;

export const regulatorOversightTargets = pgTable("regulatorOversightTargets", {
    id: serial("id").primaryKey(),
    regulatorOrgId: integer("regulatorOrgId")
        .NotNull()
        .references(() => organizations.id, { onDelete: "cascade" })),
    targetOrgId: integer("targetOrgId")
        .NotNull()
        .references(() => organizations.id, { onDelete: "cascade" })),
    grantedByAdminId: integer("grantedByAdminId").references(() => users.id),
    expiresAt: timestamp("expiresAt"),
    isActive: integer("isActive").default(1).NotNull(),
    justification: text("justification"),
    createdAt: timestamp("createdAt").defaultNow().NotNull(),
    updatedAt: timestamp("updatedAt").defaultNow().NotNull(),
});

export type RegulatorOversightTarget =
    typeof regulatorOversightTargets.$inferSelect;
export type InsertRegulatorOversightTarget =
    typeof regulatorOversightTargets.$inferInsert;

//
// COMPLIANCE EVIDENCE LOCKER
//

export const complianceEvidence = pgTable("complianceEvidence", {
    id: serial("id").primaryKey(),
    organizationId: integer("organizationId")
        .NotNull()
        .references(() => organizations.id, { onDelete: "cascade" })),
    sourceType: evidenceSourceTypeEnum("sourceType").NotNull().default("general"),
    sourceId: integer("sourceId"),
    title: varchar("title", { length: 255 }).NotNull(),
});

```

```

url: varchar("url", { length: 1024 }).notNull(),
description: text("description"),
addedByUserId: integer("addedByUserId").references(() => localUsers.id, {
  onDelete: "set null",
}),
tags: varchar("tags", { length: 512 }),
createdAt: timestamp("createdAt").defaultNow().notNull(),
updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type ComplianceEvidence = typeof complianceEvidence.$inferSelect;
export type InsertComplianceEvidence = typeof complianceEvidence.$inferInsert;

//
// DATA SUBJECT REQUEST TRACKER
//

export const dsrRequests = pgTable("dsrRequests", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id, { onDelete: "cascade" }),
  requestType: dsrRequestTypeEnum("requestType").notNull(),
  jurisdiction: dsrJurisdictionEnum("jurisdiction").notNull().default("Other"),
  requesterName: varchar("requesterName", { length: 255 }).notNull(),
  requesterEmail: varchar("requesterEmail", { length: 320 }).notNull(),
  description: text("description"),
  status: dsrStatusEnum("status").notNull().default("received"),
  priority: dsrPriorityEnum("priority").notNull().default("normal"),
  dueDate: timestamp("dueDate").notNull(),
  completedAt: timestamp("completedAt"),
  assignedToUserId: integer("assignedToUserId").references(
    () => localUsers.id,
    { onDelete: "set null" }
  ),
  notes: text("notes"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type DsrRequest = typeof dsrRequests.$inferSelect;
export type InsertDsrRequest = typeof dsrRequests.$inferInsert;

//
// SERVICE REQUESTS Pentest / Audit / SOC / Consulting
//

export const serviceRequests = pgTable("serviceRequests", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id, { onDelete: "cascade" }),
  requestedByUserId: integer("requestedByUserId").references(
    () => localUsers.id,
    { onDelete: "set null" }
  ),
  serviceType: serviceTypeEnum("serviceType").notNull(),
  title: varchar("title", { length: 255 }).notNull(),
  description: text("description").notNull(),
  scopeDetails: text("scopeDetails"),
  preferredStartDate: timestamp("preferredStartDate"),
  budgetRange: varchar("budgetRange", { length: 120 }),
  priority: servicePriorityEnum("priority").default("medium").notNull(),
  status: serviceStatusEnum("status").default("submitted").notNull(),
  assignedToUserId: integer("assignedToUserId").references(
    () => localUsers.id,
    { onDelete: "set null" }
  ),
  internalNotes: text("internalNotes"),
  clientResponse: text("clientResponse"),
  respondedAt: timestamp("respondedAt"),
  completedAt: timestamp("completedAt"),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

```

```

});

export type ServiceRequest = typeof serviceRequests.$inferSelect;
export type InsertServiceRequest = typeof serviceRequests.$inferInsert;

//
// ASSET INVENTORY
//

export const assetInventory = pgTable("assetInventory", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id, { onDelete: "cascade" }),
  name: varchar("name", { length: 255 }).notNull(),
  assetType: inventoryAssetTypeEnum("assetType").notNull(),
  identifier: varchar("identifier", { length: 512 }),
  owner: varchar("owner", { length: 255 }),
  location: varchar("location", { length: 255 }),
  criticality: criticalityEnum("criticality").default("medium").notNull(),
  exposure: exposureEnum("exposure").default("internal").notNull(),
  status: inventoryStatusEnum("status").default("active").notNull(),
  riskScore: integer("riskScore").default(0).notNull(),
  platform: varchar("platform", { length: 120 }),
  version: varchar("version", { length: 120 }),
  lastScannedAt: timestamp("lastScannedAt"),
  openVulnCount: integer("openVulnCount").default(0).notNull(),
  tags: varchar("tags", { length: 512 }),
  notes: text("notes"),
  addedByUserId: integer("addedByUserId").references(() => localUsers.id, {
    onDelete: "set null",
  }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type AssetInventoryItem = typeof assetInventory.$inferSelect;
export type InsertAssetInventoryItem = typeof assetInventory.$inferInsert;

//
// SECURITY MATURITY
//

export const securityMaturityAssessments = pgTable(
  "securityMaturityAssessments",
  {
    id: serial("id").primaryKey(),
    organizationId: integer("organizationId")
      .notNull()
      .references(() => organizations.id, { onDelete: "cascade" }),
    title: varchar("title", { length: 255 }).notNull(),
    frameworkRef: varchar("frameworkRef", { length: 64 }),
    scoreGovernance: integer("scoreGovernance").default(1).notNull(),
    scoreAssetManagement: integer("scoreAssetManagement").default(1).notNull(),
    scoreAccessControl: integer("scoreAccessControl").default(1).notNull(),
    scoreDataProtection: integer("scoreDataProtection").default(1).notNull(),
    scoreNetworkSecurity: integer("scoreNetworkSecurity").default(1).notNull(),
    scoreVulnerabilityMgmt: integer("scoreVulnerabilityMgmt")
      .default(1)
      .notNull(),
    scoreIncidentResponse: integer("scoreIncidentResponse")
      .default(1)
      .notNull(),
    scoreBackupRecovery: integer("scoreBackupRecovery").default(1).notNull(),
    scoreThirdPartyRisk: integer("scoreThirdPartyRisk").default(1).notNull(),
    scoreSecurityAwareness: integer("scoreSecurityAwareness")
      .default(1)
      .notNull(),
    overallScore: integer("overallScore").default(0).notNull(),
    maturityLevel: maturityLevelEnum("maturityLevel")
      .default("initial")
      .notNull(),
    recommendations: text("recommendations"),
    assessedByUserId: integer("assessedByUserId").references(

```

```

        () => localUsers.id,
        { onDelete: "set null" }
    ),
    createdAt: timestamp("createdAt").defaultNow().notNull(),
    updatedAt: timestamp("updatedAt").defaultNow().notNull(),
}
);

export type SecurityMaturityAssessment =
    typeof securityMaturityAssessments.$inferSelect;
export type InsertSecurityMaturityAssessment =
    typeof securityMaturityAssessments.$inferInsert;

//
// THREAT INTELLIGENCE FEED
//

export const threatIntelItems = pgTable("threatIntelItems", {
    id: serial("id").primaryKey(),
    organizationId: integer("organizationId").references(() => organizations.id, {
        onDelete: "cascade",
    }),
    title: varchar("title", { length: 255 }).notNull(),
    summary: text("summary").notNull(),
    threatActor: varchar("threatActor", { length: 180 }),
    category: threatCategoryEnum("category").notNull(),
    severity: threatSeverityEnum("severity").default("medium").notNull(),
    tlp: tlpEnum("tlp").default("white").notNull(),
    affectedSectors: varchar("affectedSectors", { length: 512 }),
    indicators: text("indicators"),
    referenceUrl: varchar("referenceUrl", { length: 1024 }),
    cveId: varchar("cveId", { length: 32 }),
    cvssScore: varchar("cvssScore", { length: 8 }),
    isActive: integer("isActive").default(1).notNull(),
    createdByUserId: integer("createdByUserId").references(() => localUsers.id, {
        onDelete: "set null",
    }),
    publishedAt: timestamp("publishedAt").defaultNow().notNull(),
    createdAt: timestamp("createdAt").defaultNow().notNull(),
    updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type ThreatIntelItem = typeof threatIntelItems.$inferSelect;
export type InsertThreatIntelItem = typeof threatIntelItems.$inferInsert;

export const yallaAdminSessions = pgTable("yallaAdminSessions", {
    id: varchar("id", { length: 64 }).primaryKey().notNull(),
    adminUsername: varchar("adminUsername", { length: 120 }).notNull(),
    ipAddress: varchar("ipAddress", { length: 64 }).notNull(),
    userAgent: varchar("userAgent", { length: 512 }),
    createdAt: timestamp("createdAt").defaultNow().notNull(),
    expiresAt: timestamp("expiresAt").notNull(),
    lastSeenAt: timestamp("lastSeenAt").defaultNow().notNull(),
    isRevoked: integer("isRevoked").default(0).notNull(),
});

export type YallaAdminSession = typeof yallaAdminSessions.$inferSelect;
export type InsertYallaAdminSession = typeof yallaAdminSessions.$inferInsert;

export const yallaAdminAuditLogs = pgTable("yallaAdminAuditLogs", {
    id: serial("id").primaryKey(),
    sessionId: varchar("sessionId", { length: 64 }),
    adminUsername: varchar("adminUsername", { length: 120 }).notNull(),
    action: varchar("action", { length: 120 }).notNull(),
    target: varchar("target", { length: 255 }),
    ipAddress: varchar("ipAddress", { length: 64 }).notNull(),
    payload: text("payload"),
    createdAt: timestamp("createdAt").defaultNow().notNull(),
});

export type YallaAdminAuditLog = typeof yallaAdminAuditLogs.$inferSelect;
export type InsertYallaAdminAuditLog = typeof yallaAdminAuditLogs.$inferInsert;

export const otpCodes = pgTable("otpCodes", {

```



```

    id: serial("id").primaryKey(),
    identifier: varchar("identifier", { length: 320 }).notNull(), // email or phone number
    codeHash: varchar("codeHash", { length: 64 }).notNull(), // SHA-256 of OTP
    purpose: varchar("purpose", { length: 32 }).notNull().default("login"), // login | register
    expiresAt: timestamp("expiresAt").notNull(),
    attempts: integer("attempts").default(0).notNull(),
    createdAt: timestamp("createdAt").defaultNow().notNull(),
  });

export type OtpCode = typeof otpCodes.$inferSelect;
export type InsertOtpCode = typeof otpCodes.$inferInsert;

//
// KNOWLEDGE GRAPH Global Compliance Knowledge Graph
//

export const knowledgeGraphNodeKindEnum = pgEnum("knowledgeGraphNodeKind", [
  "region",
  "framework",
  "standard",
  "edition",
  "agent",
  "regulator",
  "country",
  "control",
  "threat",
  "vendor",
  "certification",
  "policy",
  "technology",
  "data_type",
  "industry",
  "risk_scenario",
]);

export const knowledgeGraphEdgeRelationEnum = pgEnum(
  "knowledgeGraphEdgeRelation",
  [
    "contains",
    "activates",
    "supports",
    "maps_to",
    "requires",
    "conflicts",
    "depends_on",
    "governs",
    "references",
    "impacts",
    "mitigates",
    "translates_to",
    "equivalent_to",
    "cross_border_to",
  ]
);

export const knowledgeGraphNodes = pgTable("knowledgeGraphNodes", {
  id: serial("id").primaryKey(),
  nodeId: varchar("nodeId", { length: 120 }).notNull().unique(),
  label: varchar("label", { length: 255 }).notNull(),
  kind: knowledgeGraphNodeKindEnum("kind").notNull(),
  description: text("description"),
  region: varchar("region", { length: 120 }),
  jurisdiction: varchar("jurisdiction", { length: 120 }),
  metadata: text("metadata"),
  organizationId: integer("organizationId").references(() => organizations.id, {
    onDelete: "cascade",
  }),
  isCustom: integer("isCustom").default(0).notNull(),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type KnowledgeGraphNode = typeof knowledgeGraphNodes.$inferSelect;
export type InsertKnowledgeGraphNode = typeof knowledgeGraphNodes.$inferInsert;

```

```

export const knowledgeGraphEdges = pgTable("knowledgeGraphEdges", {
  id: serial("id").primaryKey(),
  sourceNodeId: varchar("sourceNodeId", { length: 120 }).notNull(),
  targetNodeId: varchar("targetNodeId", { length: 120 }).notNull(),
  relation: knowledgeGraphEdgeRelationEnum("relation").notNull(),
  weight: integer("weight").default(1).notNull(),
  metadata: text("metadata"),
  organizationId: integer("organizationId").references(() => organizations.id, {
    onDelete: "cascade",
  }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
});

export type KnowledgeGraphEdge = typeof knowledgeGraphEdges.$inferSelect;
export type InsertKnowledgeGraphEdge = typeof knowledgeGraphEdges.$inferInsert;

//
// AI AGENT RUNS Multi-Agent Orchestration Log
//

export const aiAgentRunStatusEnum = pgEnum("aiAgentRunStatus", [
  "queued",
  "running",
  "completed",
  "failed",
  "cancelled",
]);

export const aiAgentRuns = pgTable("aiAgentRuns", {
  id: serial("id").primaryKey(),
  organizationId: integer("organizationId")
    .notNull()
    .references(() => organizations.id, { onDelete: "cascade" }),
  agentCode: varchar("agentCode", { length: 120 }).notNull(),
  agentName: varchar("agentName", { length: 255 }).notNull(),
  triggerType: varchar("triggerType", { length: 64 })
    .default("manual")
    .notNull(),
  inputPayload: text("inputPayload"),
  outputPayload: text("outputPayload"),
  status: aiAgentRunStatusEnum("status").default("queued").notNull(),
  startedAt: timestamp("startedAt"),
  completedAt: timestamp("completedAt"),
  errorMessage: text("errorMessage"),
  durationMs: integer("durationMs"),
  createdByUserId: integer("createdByUserId").references(() => localUsers.id, {
    onDelete: "set null",
  }),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().notNull(),
});

export type AiAgentRun = typeof aiAgentRuns.$inferSelect;
export type InsertAiAgentRun = typeof aiAgentRuns.$inferInsert;

```

undefined | TypeScript

```

import {
  NOT_ADMIN_ERR_MSG,
  UNAUTHED_ERR_MSG,
  NOT_PLATFORM_ADMIN_ERR_MSG,
  NOT_SUPER_ADMIN_ERR_MSG,
  NOT_COMPANY_ADMIN_ERR_MSG,
  hasMinRole,
} from "../../shared/const";
import { initTRPC, TRPCError } from "@trpc/server";
import superjson from "superjson";
import type { TrpcContext } from "../context";
import { organizations, userOnboarding } from "../../drizzle/schema";
import { eq, or } from "drizzle-orm";

```

```

import type { Vendor } from "../../drizzle/schema";
import { complianceControls, frameworks } from "../../drizzle/schema";
import { getDb } from "../db";
import {
  runDualJurisdictionAssessment,
  type SupplierAssessmentResult,
  type SupplierGap,
} from "../supplier-assessment";
import { ENV } from "../_core/env";
import {
  aiAssessmentReportSchema,
  type AiAssessmentReport,
  type AiJobStage,
  type ExtractedFact,
  type RagControl,
  type SupplierGapPayload,
} from "../schemas";
export type PipelineEngine = "native";

type PipelineInput = {
  source: "vendor_profile" | "document_upload";
  vendor: Vendor;
  rawDocumentText?: string;
  engine?: PipelineEngine;
};

type StageReporter = (stage: AiJobStage, message: string) => void;

type IntakeResult = {
  documentType: string;
  tags: string[];
  normalizedText: string;
};

type ValidatorResult = {
  passed: boolean;
  notes: string[];
};

const WEAK_ENCRYPTION_CODE = "CRYPTO-WEAK-001";
const DEFAULT_KNOWN_FRAMEWORKS = [
  "PIPL",
  "CSL",
  "DSL",
  "MLPS 2.0",
  "PDPL",
  "NCA",
];
const CHINA_FRAMEWORKS = ["PIPL", "CSL", "DSL", "MLPS 2.0"];
const SAUDI_FRAMEWORKS = ["PDPL", "NCA"];

const INJECTION_PATTERNS: RegExp[] = [
  /ignore\s+all\s+previous\s+instructions/i,
  /system\s+prompt/i,
  /jailbreak/i,
  /<script[\s>]/i,
  /rm\s+-rf\s+\/\//i,
  /drop\s+table/i,
  /shutdown\s+-h/i,
];

const CONTROL_BUCKET_KEYWORDS: Record<string, string[]> = {
  "Data Processing & Encryption": [
    "encrypt",
    "encryption",
    "aes",
    "rsa",
    "key",
  ],
  "Data Localization": [

```

```

    "china",
    "saudi",
    "riyadh",
    "beijing",
    "shanghai",
    "localization",
  ],
  "Access Control": ["access", "identity", "iam", "mfa", "privilege"],
  "Incident Response": ["incident", "breach", "response", "monitor"],
  "Transfer & Consent": [
    "consent",
    "transfer",
    "cross-border",
    "cross border",
    "subject",
  ],
  Governance: ["audit", "policy", "governance", "compliance", "legal"],
};

function clampScore(value: number): number {
  return Math.max(0, Math.min(100, Math.round(value)));
}

function scoreToStatus(
  score: number
): "compliant" | "partial" | "non_compliant" {
  if (score >= 85) return "compliant";
  if (score >= 65) return "partial";
  return "non_compliant";
}

function inferRiskLevel(
  assessment: SupplierAssessmentResult
): "low" | "medium" | "high" | "critical" {
  const criticalCount = assessment.gaps.filter(
    g => g.severity === "critical"
  ).length;
  const highCount = assessment.gaps.filter(g => g.severity === "high").length;

  if (criticalCount > 0) return "critical";
  if (assessment.overallScore < 60 || highCount >= 2) return "high";
  if (assessment.overallScore < 80 || highCount > 0) return "medium";
  return "low";
}

function toTokens(value: string): string[] {
  return value
    .toLowerCase()
    .replace(/[\^a-z0-9\s-]/g, " ")
    .split(/\s+/)
    .map(token => token.trim())
    .filter(token => token.length >= 3);
}

function mapControlBuckets(text: string): string[] {
  const normalized = text.toLowerCase();
  const buckets: string[] = [];

  for (const [bucket, keywords] of Object.entries(CONTROL_BUCKET_KEYWORDS)) {
    if (keywords.some(keyword => normalized.includes(keyword))) {
      buckets.push(bucket);
    }
  }

  return Array.from(new Set(buckets));
}

function addFact(
  facts: ExtractedFact[],
  key: string,
  value: string,
  evidence = ""
) {
  const normalized = value.trim();

```

```

    if (!normalized) return;

    facts.push({
      key,
      value: normalized,
      evidence: evidence.trim(),
      mappedControlBuckets: mapControlBuckets(`${key} ${normalized} ${evidence}`),
    });
  }

  async function callAgentSwarm<T>({
    stagePath: string,
    payload: unknown
  }): Promise<T | null> {
    const baseUrl = ENV.agentSwarmBaseUrl;
    if (!baseUrl) return null;

    const endpoint = `${baseUrl.replace(/\/$/, "")}/${stagePath.replace(/^\/$/, "")}`;
    const abortController = new AbortController();
    const timeout = setTimeout(() => abortController.abort(), 15_000);

    try {
      const response = await fetch(endpoint, {
        method: "POST",
        headers: {
          "content-type": "application/json",
        },
        body: JSON.stringify(payload),
        signal: abortController.signal,
      });

      if (!response.ok) {
        return null;
      }

      return (await response.json()) as T;
    } catch {
      return null;
    } finally {
      clearTimeout(timeout);
    }
  }

  function runSecurityGatekeeper(payload: string) {
    const threats = INJECTION_PATTERNS.filter(pattern => pattern.test(payload));
    if (threats.length > 0) {
      throw new Error(
        "Security Gatekeeper blocked potentially malicious assessment payload."
      );
    }
  }

  function runIntake(vendor: Vendor, rawDocumentText: string): IntakeResult {
    const combinedText = [
      vendor.vendorName,
      vendor.vendorDescription || "",
      vendor.industry || "",
      vendor.businessRegistrationNumber || "",
      vendor.headquartersLocation || "",
      vendor.primaryContactName || "",
      vendor.primaryContactEmail || "",
      vendor.primaryContactRole || "",
      vendor.primaryContactPhone || "",
      vendor.serviceType || "",
      vendor.serviceScope || "",
      vendor.hostingEnvironment || "",
      vendor.operatingCountries || "",
      vendor.cloudProvider || "",
      vendor.dataLocations || "",
      vendor.regulatoryJurisdictions || "",
      vendor.certifications || "",
      vendor.dataProcessingActivities || "",
      vendor.criticalityLevel || "",
      vendor.riskTier || "",
    ];
  }

```

```

    vendor.thirdPartyDependencies || "",
    vendor.fourthPartyDependencies || "",
    rawDocumentText || ""
  ]
  .filter(Boolean)
  .join("\n")
  .trim();

const lower = combinedText.toLowerCase();
const documentType =
  lower.includes("policy") || lower.includes("procedure")
    ? "policy_document"
    : lower.includes("questionnaire")
    ? "questionnaire"
    : rawDocumentText.trim().length > 0
    ? "uploaded_text"
    : "vendor_profile";

const tags = Array.from(
  new Set(
    [
      vendor.industry,
      vendor.businessRegistrationNumber,
      vendor.serviceType,
      vendor.hostingEnvironment,
      vendor.cloudProvider,
      vendor.operatingCountries,
      vendor.dataLocations,
      vendor.regulatoryJurisdictions,
      vendor.dataProcessingActivities,
      vendor.criticalityLevel,
      vendor.riskTier,
      documentType,
    ]
    .filter(Boolean)
    .flatMap(value =>
      String(value)
        .split(/[;|]/)
        .map(v => v.trim().toLowerCase())
        .filter(Boolean)
    )
  )
).slice(0, 20);

return {
  documentType,
  tags,
  normalizedText: combinedText,
};
}

function runExtractor(vendor: Vendor, intake: IntakeResult): ExtractedFact[] {
  const facts: ExtractedFact[] = [];

  addFact(facts, "vendor_name", vendor.vendorName);
  addFact(facts, "industry", vendor.industry || "");
  addFact(
    facts,
    "business_registration_number",
    vendor.businessRegistrationNumber || ""
  );
  addFact(facts, "headquarters_location", vendor.headquartersLocation || "");
  addFact(facts, "primary_contact_name", vendor.primaryContactName || "");
  addFact(facts, "primary_contact_email", vendor.primaryContactEmail || "");
  addFact(facts, "primary_contact_role", vendor.primaryContactRole || "");
  addFact(facts, "primary_contact_phone", vendor.primaryContactPhone || "");
  addFact(facts, "service_type", vendor.serviceType || "");
  addFact(facts, "service_scope", vendor.serviceScope || "");
  addFact(facts, "hosting_environment", vendor.hostingEnvironment || "");
  addFact(facts, "cloud_provider", vendor.cloudProvider || "");
  addFact(facts, "operating_countries", vendor.operatingCountries || "");
  addFact(facts, "data_locations", vendor.dataLocations || "");
  addFact(
    facts,

```

```

    "regulatory_jurisdictions",
    vendor.regulatoryJurisdictions || ""
);
addFact(facts, "certifications", vendor.certifications || "");
addFact(
    facts,
    "data_processing_activities",
    vendor.dataProcessingActivities || ""
);
addFact(facts, "criticality_level", vendor.criticalityLevel || "");
addFact(facts, "risk_tier", vendor.riskTier || "");
addFact(
    facts,
    "third_party_dependencies",
    vendor.thirdPartyDependencies || ""
);
addFact(
    facts,
    "fourth_party_dependencies",
    vendor.fourthPartyDependencies || ""
);

const encryptionRegex = /(rsa|aes)[\s-]?(\d{3,4})/gi;
let match: RegExpExecArray | null;
while ((match = encryptionRegex.exec(intake.normalizedText))) {
    addFact(
        facts,
        "encryption_claim",
        `${match[1].toUpperCase()} ${match[2]}`,
        `Found in submission text: ${match[0]}`
    );
}

addFact(
    facts,
    "intake_document_type",
    intake.documentType,
    "Classified by Intake & Tagging Clerk"
);

return facts.slice(0, 200);
}

async function buildRagContext(facts: ExtractedFact[]): Promise<RagControl[]> {
    const db = await getDb();
    if (!db) return [];

    const frameworkRows = await db.select().from(frameworks);
    const controls = await db.select().from(complianceControls);

    const frameworkCodeById = new Map(
        frameworkRows.map(row => [row.id, row.code] as const)
    );
    const tokenSet = new Set(
        facts.flatMap(fact =>
            toTokens(`${fact.key} ${fact.value} ${fact.evidence}`)
        )
    );
};

const scored = controls
    .map(control => {
        const haystack = `${control.controlCode} ${control.controlName || ""} ${
            control.category || ""
        } ${control.requirement || ""} ${control.description || ""}`.toLowerCase();

        if (tokenSet.size === 0) {
            return {
                row: control,
                score: 0,
            };
        }
    });

let hits = 0;
tokenSet.forEach(token => {

```

```

        if (haystack.includes(token)) hits += 1;
    });

    return {
        row: control,
        score: hits / tokenSet.size,
    };
})
.filter(item => item.score > 0)
.sort((a, b) => b.score - a.score)
.slice(0, ENV.aiRagTopK);

return scored.map(item => ({
    controlId: item.row.id,
    frameworkCode: frameworkCodeById.get(item.row.frameworkId) || "UNKNOWN",
    controlCode: item.row.controlCode,
    category: item.row.category,
    controlName: item.row.controlName,
    requirement: item.row.requirement,
    relevanceScore: Number(item.score.toFixed(4)),
})));
}

function findWeakEncryptionFact(facts: ExtractedFact[]): ExtractedFact | null {
    for (const fact of facts) {
        if (fact.key !== "encryption_claim") continue;
        const match = fact.value.match(/(RSA|AES)\s*(\d{3,4})/i);
        if (!match) continue;

        const algorithm = match[1].toUpperCase();
        const bits = Number(match[2]);

        if (
            (algorithm === "RSA" && bits < 2048) ||
            (algorithm === "AES" && bits < 128)
        ) {
            return fact;
        }
    }

    return null;
}

function ensureWeakEncryptionGap(
    assessment: SupplierAssessmentResult,
    weakFact: ExtractedFact
) {
    if (assessment.gaps.some(gap => gap.code === WEAK_ENCRYPTION_CODE)) {
        return;
    }

    const newGap: SupplierGap = {
        code: WEAK_ENCRYPTION_CODE,
        jurisdiction: "cross_border",
        frameworks: ["PIPL", "PDPL", "NCA"],
        severity: "critical",
        title: "Weak encryption claim detected",
        description: `Extracted evidence indicates weak encryption posture (${weakFact.value}).`,
        mitigation:
            "Upgrade cryptographic controls to modern baseline (RSA 2048+ or AES-128+), rotate keys, and re-validate d-  

            penaltyContext:
            "Weak encryption can materially increase enforcement risk under PIPL, PDPL, and NCA security control expec~
    };

    assessment.gaps = [newGap, ...assessment.gaps];
    assessment.jurisdictionScores.china = clampScore(
        assessment.jurisdictionScores.china - 10
    );
    assessment.jurisdictionScores.saudiArabia = clampScore(
        assessment.jurisdictionScores.saudiArabia - 10
    );
    assessment.overallScore = clampScore(
        (assessment.jurisdictionScores.china +
        assessment.jurisdictionScores.saudiArabia) /

```



```

    2
  );
  assessment.status = scoreToStatus(assessment.overallScore);
  assessment.riskLevel = inferRiskLevel(assessment);
}

function runJudge(
  vendor: Vendor,
  facts: ExtractedFact[]
): SupplierAssessmentResult {
  const assessment = runDualJurisdictionAssessment(vendor);
  const weakFact = findWeakEncryptionFact(facts);

  if (weakFact) {
    ensureWeakEncryptionGap(assessment, weakFact);
  }

  const recommendationSet = new Set(assessment.recommendations);
  if (weakFact) {
    recommendationSet.add(
      "Execute emergency cryptography remediation and document validated key-management controls before producti-
    );
  }

  assessment.recommendations = Array.from(recommendationSet);
  return assessment;
}

function severityRank(severity: SupplierGapPayload["severity"]): number {
  if (severity === "critical") return 4;
  if (severity === "high") return 3;
  if (severity === "medium") return 2;
  return 1;
}

function runSynthesizer(assessment: SupplierAssessmentResult): string[] {
  const plan = [...assessment.gaps]
    .sort((a, b) => severityRank(b.severity) - severityRank(a.severity))
    .slice(0, 10)
    .map(
      gap =>
        `[${gap.severity.toUpperCase()}] ${gap.title}: ${gap.mitigation} (Frameworks: ${gap.frameworks.join(", "~
    );

  plan.push(
    "Build a jurisdiction-specific evidence register for CAC, SDAIA, and NCA review cycles.",
    "Introduce quarterly control re-validation against framework updates and supplier architecture changes.",
    "Require legal sign-off for all critical and high findings prior to vendor onboarding approval."
  );

  return Array.from(new Set(plan));
}

function _mergeExtractedFacts(
  baselineFacts: ExtractedFact[],
  additionalFacts: ExtractedFact[]
): ExtractedFact[] {
  const merged: ExtractedFact[] = [];
  const seen = new Set<string>();

  for (const fact of [...baselineFacts, ...additionalFacts]) {
    const key = `${fact.key.toLowerCase()}::${fact.value.toLowerCase()}`;
    if (seen.has(key)) {
      continue;
    }
    seen.add(key);
    merged.push(fact);

    if (merged.length >= 200) {
      break;
    }
  }

  return merged;
}

```

```

}

function _mergeRecommendations(
  existing: string[],
  additional: string[]
): string[] {
  const merged = Array.from(new Set([...additional, ...existing]));
  return merged.slice(0, 120);
}

function runValidator(
  assessment: SupplierAssessmentResult,
  remediationPlan: string[],
  ragControls: RagControl[]
): ValidatorResult {
  const knownFrameworks = new Set(
    [
      ...DEFAULT_KNOWN_FRAMEWORKS,
      ...ragControls.map(control => control.frameworkCode),
    ].map(code => code.toUpperCase())
  );

  const notes: string[] = [];
  for (const gap of assessment.gaps) {
    for (const code of gap.frameworks) {
      if (!knownFrameworks.has(code.toUpperCase())) {
        notes.push(`Unknown framework code in gap ${gap.code}: ${code}.`);
      }
    }
  }

  if (remediationPlan.length === 0) {
    notes.push("Remediation plan is empty.");
  }

  if (assessment.gaps.length > 0 && remediationPlan.length < 2) {
    notes.push("Remediation plan is too short for identified gap volume.");
  }

  return {
    passed: notes.length === 0,
    notes,
  };
}

function buildDbPayload(
  assessment: SupplierAssessmentResult,
  remediationPlan: string[],
  ragControls: RagControl[]
) {
  const frameworkAssessmentRows = [
    ...CHINA_FRAMEWORKS.map(code => ({
      frameworkCode: code,
      complianceScore: assessment.jurisdictionScores.china,
      riskLevel: assessment.riskLevel,
      status: assessment.status,
      findings: assessment.gaps
        .filter(gap => gap.frameworks.includes(code))
        .map(gap => `${gap.code}: ${gap.title}`),
      recommendations: remediationPlan,
    })),
    ...SAUDI_FRAMEWORKS.map(code => ({
      frameworkCode: code,
      complianceScore: assessment.jurisdictionScores.saudiArabia,
      riskLevel: assessment.riskLevel,
      status: assessment.status,
      findings: assessment.gaps
        .filter(gap => gap.frameworks.includes(code))
        .map(gap => `${gap.code}: ${gap.title}`),
      recommendations: remediationPlan,
    })),
  ];
}

const firstControlByFramework = new Map<string, string>();

```

```

for (const control of ragControls) {
  if (!firstControlByFramework.has(control.frameworkCode)) {
    firstControlByFramework.set(control.frameworkCode, control.controlCode);
  }
}

const gapRows = assessment.gaps.flatMap(gap =>
  gap.frameworks.map(frameworkCode => ({
    frameworkCode,
    controlCode: firstControlByFramework.get(frameworkCode) || "",
    gapCode: gap.code,
    gapDescription: gap.description,
    severity: gap.severity,
    remediation: gap.mitigation,
  })))
);

return {
  vendorAssessments: frameworkAssessmentRows,
  assessmentGaps: gapRows,
};
}

export async function executeAssessmentPipeline(
  input: PipelineInput,
  reportStage: StageReporter
): Promise<AiAssessmentReport> {
  const rawDocumentText = input.rawDocumentText?.trim() || "";
  const _requestedEngine = input.engine ?? ENV.aiAssessmentEngineDefault;

  reportStage("gatekeeper", "Security Gatekeeper scanning payload.");
  runSecurityGatekeeper(rawDocumentText);

  reportStage("intake", "Intake Clerk classifying submission.");
  const intake = runIntake(input.vendor, rawDocumentText);

  reportStage("extractor", "Extraction agent mapping facts to controls.");
  const externalFacts = await callAgentSwarm<ExtractedFact[]>({
    "frontline/extractor",
    {
      vendor: input.vendor,
      intake,
      rawDocumentText,
    }
  });
  const extractedFacts =
    externalFacts && externalFacts.length > 0
      ? externalFacts.slice(0, 200)
      : runExtractor(input.vendor, intake);

  reportStage("rag_context", "RAG context assembler retrieving controls.");
  const ragControls = await buildRagContext(extractedFacts);

  reportStage("judge", "Compliance reviewer evaluating mapped facts.");
  const externalAssessment = await callAgentSwarm<SupplierAssessmentResult>({
    "backend/judge",
    {
      vendor: input.vendor,
      extractedFacts,
      ragControls,
    }
  });
  const assessment =
    externalAssessment || runJudge(input.vendor, extractedFacts);

  reportStage(
    "synthesizer",
    "Strategic synthesizer drafting remediation plan."
  );
  let remediationPlan =
    (await callAgentSwarm<string[]>("backend/synthesizer", {
      assessment,
      ragControls,
    }))) || runSynthesizer(assessment);
}

```

```

let validatorNotes: string[] = [];
let validatorPassed = false;
let attempts = 0;
const maxAttempts = Math.max(1, ENV.aiValidatorMaxRetries + 1);

while (!validatorPassed && attempts < maxAttempts) {
  attempts += 1;
  reportStage("validator", `Validator pass ${attempts} running.`);

  const externalValidation = await callAgentSwarm<ValidatorResult>({
    "backend/validator",
    {
      assessment,
      remediationPlan,
      ragControls,
    }
  });

  const validation =
    externalValidation ||
    runValidator(assessment, remediationPlan, ragControls);
  validatorPassed = validation.passed;
  validatorNotes = validation.notes;

  if (!validatorPassed && attempts < maxAttempts) {
    remediationPlan = runSynthesizer(assessment);
  }
}

if (!validatorPassed) {
  throw new Error(
    `Validator rejected report payload after ${attempts} attempt(s): ${validatorNotes.join(
      " "
    )}`
  );
}

reportStage("reporter", "Reporter formatting strict JSON output.");
const reportPayload = {
  version: "1.0" as const,
  generatedAt: new Date().toISOString(),
  inputSummary: {
    vendorId: input.vendor.id,
    source: input.source,
    documentType: intake.documentType,
    tags: intake.tags,
  },
  extractedFacts,
  ragControls,
  assessment,
  remediationPlan,
  validator: {
    passed: validatorPassed,
    attempts,
    notes: validatorNotes,
  },
  dbPayload: buildDbPayload(assessment, remediationPlan, ragControls),
};

return aiAssessmentReportSchema.parse(reportPayload);
}

```

undefined | TypeScript

```

import { ENV } from "../_core/env";
import { executeAssessmentPipeline } from "../pipeline";
import { persistAssessmentReport } from "../persistence";
import type {
  AssessmentHistoryClearResult,
  AssessmentHistoryDiagnostics,

```

```

    AssessmentJobInput,
    QueueSnapshotListener,
  } from "./queue";
import { getAssessmentQueue } from "./queueFactory";

const queue = getAssessmentQueue();

queue.setWorker(async (job, onProgress) => {
  const report = await executeAssessmentPipeline(
    {
      source: job.source,
      engine: job.engine,
      vendor: job.vendor,
      rawDocumentText: job.rawDocumentText,
    },
    (stage, message) => {
      onProgress({ stage, message });
    }
  );

  onProgress({
    stage: "persistence",
    message: "Persisting report payload to compliance tables.",
  });

  const persistence = await persistAssessmentReport(
    report,
    Boolean(job.persistResult)
  );

  return {
    report,
    persistence,
  };
});

export function enqueueAssessmentJob(input: AssessmentJobInput) {
  return queue.enqueue(input);
}

export function getAssessmentJob(jobId: string) {
  return queue.get(jobId);
}

export function listAssessmentJobsForUser(userId: number, limit = 20) {
  return queue.listByUser(userId, limit);
}

export function getAssessmentHistoryDiagnostics(
  userId: number
): Promise<AssessmentHistoryDiagnostics> {
  return queue.getHistoryDiagnostics(userId);
}

export function clearAssessmentHistory(
  userId: number
): Promise<AssessmentHistoryClearResult> {
  return queue.clearHistory(userId);
}

export async function waitForAssessmentJob(jobId: string, timeoutMs?: number) {
  return queue.waitForCompletion(jobId, timeoutMs ?? ENV.aiJobTimeoutMs);
}

export function subscribeAssessmentJobSnapshots(
  listener: QueueSnapshotListener
) {
  return queue.subscribe(listener);
}

export async function runAssessmentSync(input: AssessmentJobInput) {
  const queued = await enqueueAssessmentJob(input);
  const finished = await waitForAssessmentJob(queued.id, ENV.aiJobTimeoutMs);
}

```

```

if (!finished) {
  throw new Error("Assessment job could not be retrieved after enqueue.");
}

if (finished.status === "failed") {
  throw new Error(
    finished.error || "Assessment orchestration failed without details."
  );
}

if (finished.status !== "completed" || !finished.result) {
  throw new Error("Assessment orchestration timed out before completion.");
}

return {
  job: finished,
  report: finished.result,
};
}

```

undefined | TypeScript

```

import { z } from "zod";

export const aiJobStatusSchema = z.enum([
  "queued",
  "running",
  "completed",
  "failed",
]);

export const aiJobStageSchema = z.enum([
  "queued",
  "gatekeeper",
  "intake",
  "extractor",
  "rag_context",
  "judge",
  "synthesizer",
  "validator",
  "reporter",
  "persistence",
  "completed",
  "failed",
]);

export const assessmentSeveritySchema = z.enum([
  "critical",
  "high",
  "medium",
  "low",
]);

export const supplierGapSchema = z.object({
  code: z.string().trim().min(1).max(120),
  jurisdiction: z.enum(["china", "saudi", "cross_border"]),
  frameworks: z.array(z.string().trim().min(1).max(32)).min(1).max(8),
  severity: assessmentSeveritySchema,
  title: z.string().trim().min(1).max(240),
  description: z.string().trim().min(1).max(3000),
  mitigation: z.string().trim().min(1).max(3000),
  penaltyContext: z.string().trim().max(2000),
});

export const supplierAssessmentSchema = z.object({
  vendorId: z.number().int(),
  generatedAt: z.string().datetime({ offset: true }),
  overallScore: z.number().int().min(0).max(100),
  jurisdictionScores: z.object({
    china: z.number().int().min(0).max(100),
    saudiArabia: z.number().int().min(0).max(100),
  }),
});

```

```

    }},
    status: z.enum(["compliant", "partial", "non_compliant"]),
    riskLevel: z.enum(["low", "medium", "high", "critical"]),
    gaps: z.array(supplierGapSchema),
    recommendations: z.array(z.string().trim().min(1).max(3000)).max(120),
  });

export const extractedFactSchema = z.object({
  key: z.string().trim().min(1).max(120),
  value: z.string().trim().min(1).max(4000),
  evidence: z.string().trim().max(4000).optional().default(""),
  mappedControlBuckets: z
    .array(z.string().trim().min(1).max(120))
    .max(12)
    .default([]),
});

export const ragControlSchema = z.object({
  controlId: z.number().int().positive(),
  frameworkCode: z.string().trim().min(1).max(32),
  controlCode: z.string().trim().min(1).max(120),
  category: z.string().trim().max(120).optional().nullable(),
  controlName: z.string().trim().max(512).optional().nullable(),
  requirement: z.string().trim().max(4000).optional().nullable(),
  relevanceScore: z.number().min(0).max(1),
});

export const dbAssessmentPayloadSchema = z.object({
  frameworkCode: z.string().trim().min(1).max(32),
  complianceScore: z.number().int().min(0).max(100),
  riskLevel: z.enum(["low", "medium", "high", "critical"]),
  status: z.enum(["compliant", "partial", "non_compliant"]),
  findings: z.array(z.string().trim().min(1).max(1200)).max(120),
  recommendations: z.array(z.string().trim().min(1).max(3000)).max(120),
});

export const dbGapPayloadSchema = z.object({
  frameworkCode: z.string().trim().min(1).max(32),
  controlCode: z.string().trim().max(120).optional().default(""),
  gapCode: z.string().trim().min(1).max(120),
  gapDescription: z.string().trim().min(1).max(3000),
  severity: assessmentSeveritySchema,
  remediation: z.string().trim().min(1).max(3000),
});

export const aiAssessmentReportSchema = z.object({
  version: z.literal("1.0"),
  generatedAt: z.string().datetime({ offset: true }),
  inputSummary: z.object({
    vendorId: z.number().int(),
    source: z.enum(["vendor_profile", "document_upload"]),
    documentType: z.string().trim().max(120),
    tags: z.array(z.string().trim().min(1).max(60)).max(20),
  }),
  extractedFacts: z.array(extractedFactSchema).max(200),
  ragControls: z.array(ragControlSchema).max(200),
  assessment: supplierAssessmentSchema,
  remediationPlan: z.array(z.string().trim().min(1).max(3000)).max(120),
  validator: z.object({
    passed: z.boolean(),
    attempts: z.number().int().min(1).max(10),
    notes: z.array(z.string().trim().min(1).max(1200)).max(60),
  }),
  dbPayload: z.object({
    vendorAssessments: z.array(dbAssessmentPayloadSchema).max(20),
    assessmentGaps: z.array(dbGapPayloadSchema).max(400),
  }),
});

export const aiJobEventSchema = z.object({
  stage: aiJobStageSchema,
  message: z.string().trim().min(1).max(500),
  timestamp: z.string().datetime({ offset: true }),
});

```

```

export const aiJobSnapshotSchema = z.object({
  id: z.string().trim().min(1),
  userId: z.number().int(),
  status: aiJobStatusSchema,
  stage: aiJobStageSchema,
  createdAt: z.string().datetime({ offset: true }),
  updatedAt: z.string().datetime({ offset: true }),
  events: z.array(aiJobEventSchema).max(500),
  error: z.string().trim().max(3000).optional(),
  result: aiAssessmentReportSchema.optional(),
  persistence: z
    .object({
      savedAssessments: z.number().int().min(0),
      savedGaps: z.number().int().min(0),
      skipped: z.boolean(),
    })
    .optional(),
});

export type AiJobStatus = z.infer<typeof aiJobStatusSchema>;
export type AiJobStage = z.infer<typeof aiJobStageSchema>;
export type SupplierGapPayload = z.infer<typeof supplierGapSchema>;
export type SupplierAssessmentPayload = z.infer<
  typeof supplierAssessmentSchema
>;
export type ExtractedFact = z.infer<typeof extractedFactSchema>;
export type RagControl = z.infer<typeof ragControlSchema>;
export type AiAssessmentReport = z.infer<typeof aiAssessmentReportSchema>;
export type AiJobEvent = z.infer<typeof aiJobEventSchema>;
export type AiJobSnapshot = z.infer<typeof aiJobSnapshotSchema>;

```

undefined | TypeScript

```

export type GlobalRegion =
  | "North America"
  | "Europe"
  | "Middle East"
  | "Asia-Pacific"
  | "Africa"
  | "Latin America"
  | "Global Standards";

export type GlobalFrameworkPack = {
  code: string;
  name: string;
  region: GlobalRegion;
  jurisdiction: string;
  category: string;
  description: string;
  scope: string;
  authority: string;
};

export type IndustryEdition = {
  code: string;
  name: string;
  sector: string;
  description: string;
  defaultFrameworkCodes: string[];
  defaultAgentCodes: string[];
};

export type AIAgentCatalogEntry = {
  code: string;
  name: string;
  focus: string;
  regions: GlobalRegion[];
};

export type GraphNodeKind =

```



```

    | "region"
    | "framework"
    | "edition"
    | "agent"
    | "standard";

export type GraphNode = {
    id: string;
    label: string;
    kind: GraphNodeKind;
    region?: GlobalRegion;
};

export type GraphEdge = {
    source: string;
    target: string;
    relation: "contains" | "activates" | "supports" | "maps_to";
};

export type GlobalRegistrySummary = {
    regions: number;
    jurisdictions: number;
    categories: number;
    authorities: number;
    frameworks: number;
    editions: number;
    agents: number;
    graphNodes: number;
    graphEdges: number;
};

export const GLOBAL_REGIONS: GlobalRegion[] = [
    "North America",
    "Europe",
    "Middle East",
    "Asia-Pacific",
    "Africa",
    "Latin America",
    "Global Standards",
];

export const GLOBAL_FRAMEWORK_PACKS: GlobalFrameworkPack[] = [
    {
        code: "NIST-CSF-2",
        name: "NIST Cybersecurity Framework 2.0",
        region: "North America",
        jurisdiction: "United States",
        category: "cybersecurity",
        description:
            "Framework for enterprise cybersecurity governance, risk management, and continuous improvement.",
        scope: "Organizations of all sizes across sectors.",
        authority: "NIST",
    },
    {
        code: "NIST-SP-800-53",
        name: "NIST SP 800-53",
        region: "North America",
        jurisdiction: "United States",
        category: "security controls",
        description:
            "Security and privacy control catalog for federal information systems and organizations.",
        scope: "Federal systems and regulated suppliers.",
        authority: "NIST",
    },
    {
        code: "NIST-SP-800-171",
        name: "NIST SP 800-171",
        region: "North America",
        jurisdiction: "United States",
        category: "cybersecurity",
        description:
            "Protecting controlled unclassified information in nonfederal systems.",
        scope: "Defense industrial base and suppliers.",
        authority: "NIST",
    },
];

```

```

},
{
  code: "NIST-SP-800-61",
  name: "NIST SP 800-61",
  region: "North America",
  jurisdiction: "United States",
  category: "incident response",
  description:
    "Incident handling lifecycle guidance for detection, response, and recovery.",
  scope: "Enterprise incident response programs.",
  authority: "NIST",
},
{
  code: "NIST-SP-800-207",
  name: "NIST SP 800-207 Zero Trust",
  region: "North America",
  jurisdiction: "United States",
  category: "zero trust",
  description:
    "Zero trust architecture principles for identity-centric security.",
  scope: "Cloud and enterprise identity architectures.",
  authority: "NIST",
},
{
  code: "NIST-AI-RMF",
  name: "NIST AI Risk Management Framework",
  region: "North America",
  jurisdiction: "United States",
  category: "ai governance",
  description:
    "Risk management guidance for AI systems across design, deployment, and monitoring.",
  scope: "AI developers, deployers, and operators.",
  authority: "NIST",
},
{
  code: "HIPAA",
  name: "Health Insurance Portability and Accountability Act",
  region: "North America",
  jurisdiction: "United States",
  category: "privacy",
  description:
    "Protected health information privacy and security requirements.",
  scope: "Covered entities and business associates.",
  authority: "HHS OCR",
},
{
  code: "HITECH",
  name: "Health Information Technology for Economic and Clinical Health Act",
  region: "North America",
  jurisdiction: "United States",
  category: "privacy",
  description:
    "Strengthens HIPAA enforcement and breach notification duties.",
  scope: "Health data ecosystem participants.",
  authority: "HHS OCR",
},
{
  code: "GLBA",
  name: "Gramm-Leach-Bliley Act",
  region: "North America",
  jurisdiction: "United States",
  category: "financial privacy",
  description:
    "Customer information safeguards and privacy obligations for financial institutions.",
  scope: "Financial institutions and service providers.",
  authority: "FTC / Federal Regulators",
},
{
  code: "SOX",
  name: "Sarbanes-Oxley Act",
  region: "North America",
  jurisdiction: "United States",
  category: "financial controls",
  description:

```

```

    "Public company internal controls, certification, and audit requirements.",
    scope: "Public issuers and auditors.",
    authority: "SEC / PCAOB",
  },
  {
    code: "SEC-CYBER",
    name: "SEC Cybersecurity Disclosure Rules",
    region: "North America",
    jurisdiction: "United States",
    category: "disclosure",
    description:
      "Public company cyber incident disclosure and risk governance expectations.",
    scope: "SEC registrants.",
    authority: "SEC",
  },
  {
    code: "FTC-SAFEGUARDS",
    name: "FTC Safeguards Rule",
    region: "North America",
    jurisdiction: "United States",
    category: "privacy",
    description:
      "Administrative, technical, and physical safeguards for customer information.",
    scope: "Financial institutions under FTC jurisdiction.",
    authority: "FTC",
  },
  {
    code: "CMMC",
    name: "Cybersecurity Maturity Model Certification",
    region: "North America",
    jurisdiction: "United States",
    category: "supply chain",
    description: "Defense supplier cybersecurity maturity requirements.",
    scope: "Defense contractors and subcontractors.",
    authority: "DoD",
  },
  {
    code: "FEDRAMP",
    name: "FedRAMP",
    region: "North America",
    jurisdiction: "United States",
    category: "cloud security",
    description:
      "Federal cloud authorization and continuous monitoring program.",
    scope: "Cloud service providers serving U.S. federal agencies.",
    authority: "FedRAMP PMO",
  },
  {
    code: "CJIS",
    name: "CJIS Security Policy",
    region: "North America",
    jurisdiction: "United States",
    category: "public safety",
    description:
      "Security requirements for criminal justice information handling.",
    scope: "Law enforcement and justice ecosystem.",
    authority: "FBI",
  },
  {
    code: "IRS-1075",
    name: "IRS Publication 1075",
    region: "North America",
    jurisdiction: "United States",
    category: "tax data",
    description: "Safeguards for federal tax information.",
    scope: "Entities handling federal tax data.",
    authority: "IRS",
  },
  {
    code: "PCI-DSS",
    name: "PCI DSS",
    region: "North America",
    jurisdiction: "United States",
    category: "payment security",

```

```

description: "Cardholder data security standard.",
scope: "Payment card ecosystem participants.",
authority: "PCI SSC",
},
{
  code: "SOC1",
  name: "SOC 1",
  region: "North America",
  jurisdiction: "United States",
  category: "assurance",
  description: "Assurance on controls relevant to financial reporting.",
  scope: "Service organizations.",
  authority: "AICPA",
},
{
  code: "SOC2",
  name: "SOC 2",
  region: "North America",
  jurisdiction: "United States",
  category: "assurance",
  description:
    "Assurance on security, availability, processing integrity, confidentiality, and privacy.",
  scope: "Service organizations.",
  authority: "AICPA",
},
{
  code: "SOC3",
  name: "SOC 3",
  region: "North America",
  jurisdiction: "United States",
  category: "assurance",
  description: "Public-facing assurance report for general trust principles.",
  scope: "Public trust reporting.",
  authority: "AICPA",
},
{
  code: "CIS-V8",
  name: "CIS Controls v8",
  region: "North America",
  jurisdiction: "United States",
  category: "security controls",
  description:
    "Prioritized cybersecurity safeguards across core control families.",
  scope: "Enterprise and small-to-large organizations.",
  authority: "CIS",
},
{
  code: "COBIT-2019",
  name: "COBIT 2019",
  region: "North America",
  jurisdiction: "Global",
  category: "governance",
  description:
    "Enterprise governance and management objectives for information and technology.",
  scope: "Governance and assurance programs.",
  authority: "ISACA",
},
{
  code: "PIPEDA",
  name: "Personal Information Protection and Electronic Documents Act",
  region: "North America",
  jurisdiction: "Canada",
  category: "privacy",
  description:
    "Canadian private-sector privacy law for personal information processing.",
  scope: "Private-sector organizations in Canada.",
  authority: "Office of the Privacy Commissioner of Canada",
},
{
  code: "CCCS-GUIDANCE",
  name: "Canadian Centre for Cyber Security Guidance",
  region: "North America",
  jurisdiction: "Canada",
  category: "cybersecurity",

```

```

description: "National cybersecurity guidance, baselines, and advisories.",
scope: "Canadian organizations and critical sectors.",
authority: "Canadian Centre for Cyber Security",
},
{
  code: "GDPR",
  name: "General Data Protection Regulation",
  region: "Europe",
  jurisdiction: "European Union",
  category: "privacy",
  description:
    "EU data protection regulation for personal data processing and rights.",
  scope: "Controllers and processors in the EU or targeting EU residents.",
  authority: "European Commission",
},
{
  code: "UK-GDPR",
  name: "UK GDPR",
  region: "Europe",
  jurisdiction: "United Kingdom",
  category: "privacy",
  description: "UK retained data protection framework aligned to GDPR.",
  scope: "UK data controllers and processors.",
  authority: "ICO",
},
{
  code: "NIS2",
  name: "NIS2 Directive",
  region: "Europe",
  jurisdiction: "European Union",
  category: "cybersecurity",
  description:
    "Security and incident management obligations for essential and important entities.",
  scope: "Critical and important entities across the EU.",
  authority: "ENISA / Member States",
},
{
  code: "DORA",
  name: "Digital Operational Resilience Act",
  region: "Europe",
  jurisdiction: "European Union",
  category: "financial resilience",
  description: "ICT risk management and resilience for financial entities.",
  scope: "EU financial sector.",
  authority: "European Supervisory Authorities",
},
{
  code: "CRA",
  name: "Cyber Resilience Act",
  region: "Europe",
  jurisdiction: "European Union",
  category: "product security",
  description:
    "Cybersecurity requirements for products with digital elements.",
  scope: "Manufacturers, importers, and distributors.",
  authority: "European Commission",
},
{
  code: "EU-AI-ACT",
  name: "EU AI Act",
  region: "Europe",
  jurisdiction: "European Union",
  category: "ai governance",
  description: "Risk-based regulation for artificial intelligence systems.",
  scope: "AI providers, deployers, importers, and distributors.",
  authority: "European Union",
},
{
  code: "EPD",
  name: "ePrivacy Directive",
  region: "Europe",
  jurisdiction: "European Union",
  category: "privacy",
  description: "Privacy and electronic communications rules.",

```

```

    scope: "Electronic communications and tracking technologies.",
    authority: "European Union",
  },
  {
    code: "PSD2",
    name: "Payment Services Directive 2",
    region: "Europe",
    jurisdiction: "European Union",
    category: "financial services",
    description:
      "Payment security, authentication, and open banking obligations.",
    scope: "Payment service providers.",
    authority: "European Union",
  },
  {
    code: "DSA",
    name: "Digital Services Act",
    region: "Europe",
    jurisdiction: "European Union",
    category: "platform governance",
    description:
      "Rules for online intermediary services and systemic risk management.",
    scope: "Digital service providers.",
    authority: "European Union",
  },
  {
    code: "DMA",
    name: "Digital Markets Act",
    region: "Europe",
    jurisdiction: "European Union",
    category: "platform governance",
    description: "Gatekeeper obligations for core platform services.",
    scope: "Large online platforms.",
    authority: "European Union",
  },
  {
    code: "ENISA-GUIDANCE",
    name: "ENISA Guidance",
    region: "Europe",
    jurisdiction: "European Union",
    category: "cybersecurity",
    description:
      "European Union agency guidance on cybersecurity and resilience.",
    scope: "EU cybersecurity programs.",
    authority: "ENISA",
  },
  {
    code: "ISO-27001",
    name: "ISO/IEC 27001",
    region: "Europe",
    jurisdiction: "Global",
    category: "information security",
    description: "Information security management system standard.",
    scope: "Organizational ISMS programs.",
    authority: "ISO/IEC",
  },
  {
    code: "ISO-27017",
    name: "ISO/IEC 27017",
    region: "Europe",
    jurisdiction: "Global",
    category: "cloud security",
    description: "Cloud security controls guidance.",
    scope: "Cloud providers and customers.",
    authority: "ISO/IEC",
  },
  {
    code: "ISO-27018",
    name: "ISO/IEC 27018",
    region: "Europe",
    jurisdiction: "Global",
    category: "privacy",
    description:
      "Protection of personally identifiable information in public clouds.",
  }

```

```

    scope: "Public cloud PII processors.",
    authority: "ISO/IEC",
  },
  {
    code: "ISO-27701",
    name: "ISO/IEC 27701",
    region: "Europe",
    jurisdiction: "Global",
    category: "privacy",
    description: "Privacy information management extension to ISO 27001/27002.",
    scope: "Privacy governance programs.",
    authority: "ISO/IEC",
  },
  {
    code: "ISO-22301",
    name: "ISO/IEC 22301",
    region: "Europe",
    jurisdiction: "Global",
    category: "resilience",
    description: "Business continuity management system standard.",
    scope: "Continuity and resilience programs.",
    authority: "ISO/IEC",
  },
  {
    code: "ISO-42001",
    name: "ISO/IEC 42001",
    region: "Europe",
    jurisdiction: "Global",
    category: "ai governance",
    description: "AI management system standard.",
    scope: "AI governance programs.",
    authority: "ISO/IEC",
  },
  {
    code: "ISO-31000",
    name: "ISO 31000",
    region: "Europe",
    jurisdiction: "Global",
    category: "risk management",
    description: "Risk management principles and guidelines.",
    scope: "Enterprise risk management.",
    authority: "ISO",
  },
  {
    code: "PDPL-KSA",
    name: "Saudi Personal Data Protection Law",
    region: "Middle East",
    jurisdiction: "Saudi Arabia",
    category: "privacy",
    description:
      "Saudi national privacy law for personal data processing and rights.",
    scope: "Controllers and processors handling personal data in Saudi Arabia.",
    authority: "SDAIA",
  },
  {
    code: "NCA-ECC",
    name: "NCA Essential Cybersecurity Controls",
    region: "Middle East",
    jurisdiction: "Saudi Arabia",
    category: "cybersecurity",
    description:
      "Mandatory baseline cybersecurity controls for Saudi entities.",
    scope: "Government, critical sectors, and regulated organizations.",
    authority: "NCA",
  },
  {
    code: "NCA-CCC",
    name: "NCA Cloud Cybersecurity Controls",
    region: "Middle East",
    jurisdiction: "Saudi Arabia",
    category: "cloud security",
    description:
      "Cloud-specific cybersecurity controls and assurance requirements.",
    scope: "Cloud service providers and tenants in KSA.",
  }

```

```

    authority: "NCA",
  },
  {
    code: "ECC-1",
    name: "ECC-1",
    region: "Middle East",
    jurisdiction: "Saudi Arabia",
    category: "cybersecurity",
    description:
      "Saudi cybersecurity control baseline identifier used by regulated entities.",
    scope: "Entities under the Saudi control baseline.",
    authority: "NCA",
  },
  {
    code: "KSA-ECC",
    name: "Essential Cybersecurity Controls",
    region: "Middle East",
    jurisdiction: "Saudi Arabia",
    category: "cybersecurity",
    description:
      "Alias for the Saudi cybersecurity control baseline used in organizational mapping.",
    scope: "Regulated Saudi organizations.",
    authority: "NCA",
  },
  {
    code: "CST-CLOUD",
    name: "CST Cloud Regulations",
    region: "Middle East",
    jurisdiction: "Saudi Arabia",
    category: "cloud regulation",
    description: "Saudi cloud regulations and cloud service requirements.",
    scope: "Cloud providers and cloud tenants.",
    authority: "CST",
  },
  {
    code: "UAE-PDPL",
    name: "UAE Personal Data Protection Law",
    region: "Middle East",
    jurisdiction: "United Arab Emirates",
    category: "privacy",
    description:
      "UAE federal privacy law for personal data processing and transfers.",
    scope: "Controllers and processors across the UAE.",
    authority: "UAE Data Office",
  },
  {
    code: "DESC-ISR",
    name: "DESC Information Security Regulation",
    region: "Middle East",
    jurisdiction: "United Arab Emirates",
    category: "cybersecurity",
    description:
      "Dubai electronic security and information protection requirements.",
    scope: "Dubai government and related entities.",
    authority: "DESC",
  },
  {
    code: "UAE-IA",
    name: "UAE Information Assurance Standards",
    region: "Middle East",
    jurisdiction: "United Arab Emirates",
    category: "cybersecurity",
    description: "National information assurance standards for UAE entities.",
    scope: "Public and regulated sectors.",
    authority: "UAE National Cybersecurity Council",
  },
  {
    code: "QCB-FRAMEWORK",
    name: "Qatar Central Bank Framework",
    region: "Middle East",
    jurisdiction: "Qatar",
    category: "financial services",
    description:
      "Cybersecurity and risk framework for Qatari financial institutions.",
  }

```



```

    scope: "Banks and payment ecosystem participants.",
    authority: "QCB",
  },
  {
    code: "QATAR-NIA",
    name: "Qatar National Information Assurance",
    region: "Middle East",
    jurisdiction: "Qatar",
    category: "cybersecurity",
    description: "National assurance requirements for information systems.",
    scope: "Government and critical sectors.",
    authority: "Qatar National Cyber Security Agency",
  },
  {
    code: "BHR-PDPL",
    name: "Bahrain Personal Data Protection Law",
    region: "Middle East",
    jurisdiction: "Bahrain",
    category: "privacy",
    description: "Bahraini data protection law for personal data processing.",
    scope: "Controllers and processors in Bahrain.",
    authority: "Bahrain Personal Data Protection Authority",
  },
  {
    code: "OMAN-CYBER",
    name: "Oman National Cybersecurity Requirements",
    region: "Middle East",
    jurisdiction: "Oman",
    category: "cybersecurity",
    description: "Omani cybersecurity control and governance requirements.",
    scope: "Government and regulated sectors.",
    authority: "National Cybersecurity Centre",
  },
  {
    code: "KUWAIT-CYBER",
    name: "Kuwait National Cybersecurity Regulations",
    region: "Middle East",
    jurisdiction: "Kuwait",
    category: "cybersecurity",
    description: "Kuwaiti cybersecurity obligations for regulated entities.",
    scope: "Public and regulated organizations.",
    authority: "Kuwait National Cyber Security Center",
  },
  {
    code: "MLPS-2",
    name: "MLPS 2.0",
    region: "Asia-Pacific",
    jurisdiction: "China",
    category: "cybersecurity",
    description:
      "China's multi-level protection scheme for information systems.",
    scope: "Networked information systems in China.",
    authority: "MPS / CAC",
  },
  {
    code: "CHINA-CRYPT",
    name: "Cryptography Law",
    region: "Asia-Pacific",
    jurisdiction: "China",
    category: "cryptography",
    description: "Chinese cryptography governance and product requirements.",
    scope: "Cryptography products and regulated users.",
    authority: "CAC / State Cryptography Administration",
  },
  {
    code: "CHINA-AI",
    name: "AI Regulations",
    region: "Asia-Pacific",
    jurisdiction: "China",
    category: "ai governance",
    description:
      "Chinese requirements for generative AI, algorithmic recommendation, and AI governance.",
    scope: "AI providers and deployers in China.",
    authority: "CAC",
  }

```

```

},
{
  code: "PDPA-SG",
  name: "Singapore Personal Data Protection Act",
  region: "Asia-Pacific",
  jurisdiction: "Singapore",
  category: "privacy",
  description: "Singapore privacy and data protection obligations.",
  scope: "Organizations processing personal data in Singapore.",
  authority: "PDPC",
},
{
  code: "MAS-TRM",
  name: "MAS Technology Risk Management",
  region: "Asia-Pacific",
  jurisdiction: "Singapore",
  category: "financial services",
  description:
    "Technology risk management for Singapore financial institutions.",
  scope: "MAS-regulated financial entities.",
  authority: "Monetary Authority of Singapore",
},
{
  code: "MAS-NOTICES",
  name: "MAS Notices",
  region: "Asia-Pacific",
  jurisdiction: "Singapore",
  category: "financial services",
  description:
    "Regulatory notices governing banking, payment, and resilience obligations.",
  scope: "MAS-regulated institutions.",
  authority: "Monetary Authority of Singapore",
},
{
  code: "APPI",
  name: "Act on the Protection of Personal Information",
  region: "Asia-Pacific",
  jurisdiction: "Japan",
  category: "privacy",
  description: "Japan's personal data protection law.",
  scope: "Controllers and processors handling Japanese personal information.",
  authority: "PPC",
},
{
  code: "PIPA-KR",
  name: "Personal Information Protection Act",
  region: "Asia-Pacific",
  jurisdiction: "South Korea",
  category: "privacy",
  description: "South Korean privacy and personal data processing law.",
  scope: "Personal information controllers and processors.",
  authority: "PIPC",
},
{
  code: "ESSENTIAL-EIGHT",
  name: "Essential Eight",
  region: "Asia-Pacific",
  jurisdiction: "Australia",
  category: "cybersecurity",
  description:
    "Australian cyber mitigation strategies for common attack paths.",
  scope: "Australian government and regulated organizations.",
  authority: "ACSC",
},
{
  code: "ISM-AU",
  name: "Australian Information Security Manual",
  region: "Asia-Pacific",
  jurisdiction: "Australia",
  category: "cybersecurity",
  description:
    "Australian cyber security guidance and controls for government and industry.",
  scope: "Australian entities handling sensitive systems.",
  authority: "ACSC",
}

```

```

},
{
  code: "PRIVACY-ACT-AU",
  name: "Australian Privacy Act",
  region: "Asia-Pacific",
  jurisdiction: "Australia",
  category: "privacy",
  description: "Australian privacy and personal information obligations.",
  scope: "Australian organizations and APP entities.",
  authority: "OAIC",
},
{
  code: "NZ-PRIVACY",
  name: "New Zealand Privacy Act",
  region: "Asia-Pacific",
  jurisdiction: "New Zealand",
  category: "privacy",
  description: "New Zealand privacy law and information privacy principles.",
  scope: "Organizations handling personal information in New Zealand.",
  authority: "Office of the Privacy Commissioner",
},
{
  code: "DPDP-IN",
  name: "Digital Personal Data Protection Act",
  region: "Asia-Pacific",
  jurisdiction: "India",
  category: "privacy",
  description: "India's personal data protection law.",
  scope: "Data fiduciaries and processors in India.",
  authority: "Data Protection Board of India",
},
{
  code: "CERT-IN",
  name: "CERT-In Directions",
  region: "Asia-Pacific",
  jurisdiction: "India",
  category: "incident response",
  description:
    "Indian cyber incident reporting and log retention directions.",
  scope: "Critical sectors and regulated organizations.",
  authority: "CERT-In",
},
{
  code: "MY-PDPA",
  name: "Malaysia Personal Data Protection Act",
  region: "Asia-Pacific",
  jurisdiction: "Malaysia",
  category: "privacy",
  description: "Malaysia's personal data protection requirements.",
  scope: "Commercial organizations in Malaysia.",
  authority: "JPDPA",
},
{
  code: "ID-PDP",
  name: "Indonesia Personal Data Protection Law",
  region: "Asia-Pacific",
  jurisdiction: "Indonesia",
  category: "privacy",
  description: "Indonesia's personal data protection framework.",
  scope: "Controllers and processors in Indonesia.",
  authority: "Ministry of Communication and Digital Affairs",
},
{
  code: "TH-PDPA",
  name: "Thailand Personal Data Protection Act",
  region: "Asia-Pacific",
  jurisdiction: "Thailand",
  category: "privacy",
  description: "Thailand's personal data protection law.",
  scope: "Organizations handling Thai personal data.",
  authority: "PDPC Thailand",
},
{
  code: "VN-PDPD",

```